22 CSC_5AI06_TP - Deep Learning I

22.1 Multilayer Perceptron (MLP)

A **Multilayer Perceptron (MLP)** is a feed-forward neural network composed of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of its inputs, adds a bias, and applies a non-linear activation function.

For layer l, the forward propagation is defined as:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \qquad a^{[l]} = g^{[l]}(z^{[l]})$$

where: $W^{[l]}$ is the weight matrix of layer l, $b^{[l]}$ is the bias vector, $z^{[l]}$ is the pre-activation, $a^{[l]}$ is the activation, $g^{[l]}$ is the activation function.

The complete learning pipeline can be summarized as: **forward propagation** → **loss computation** → **backpropagation** → **parameter update**.

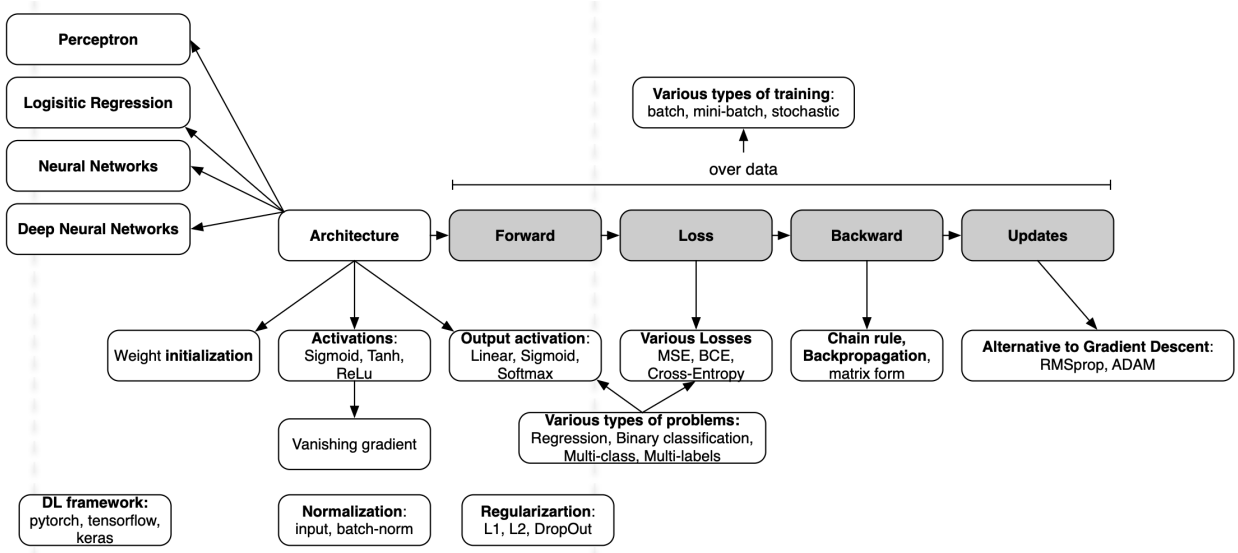


Figure 4: MLP and Deep Learning pipeline with the differents tuning methods

22.1.1 Backpropagation

Backpropagation is an efficient algorithm for computing the gradients of the loss function with respect to all network parameters.

The main idea is to apply the **chain rule** recursively, decomposing a complex derivative into a sequence of local derivatives. Instead of computing each derivative independently, intermediate quantities computed during the forward pass are reused during the backward pass.

For a parameter θ:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial a^{[L]}} \frac{\partial a^{[L]}}{\partial z^{[L]}} \cdots \frac{\partial z^{[l]}}{\partial \theta}$$

This approach provides:

- efficient computation of gradients
- reuse of intermediate values
- reduced computational cost compared with computing each derivative independently
- a memory/computation trade-off, since intermediate activations can be stored and reused

The forward pass computes the network output, while the backward pass propagates the error from the output layer towards the input layer.

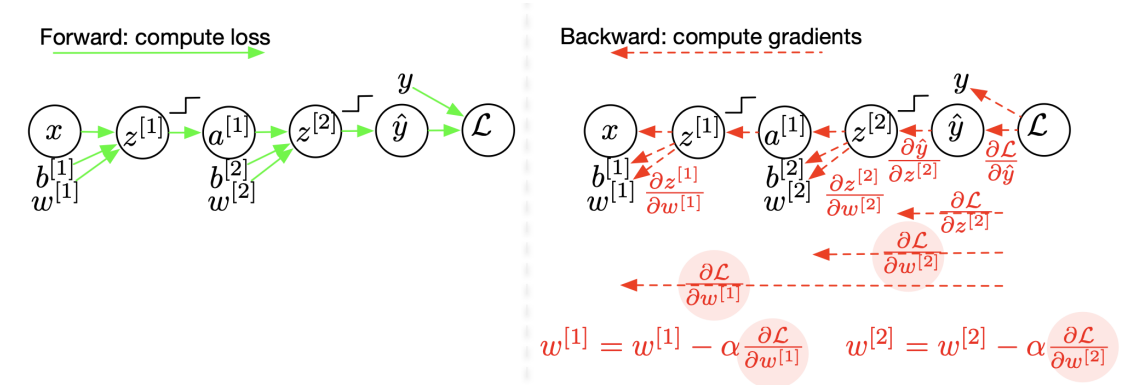


Figure 5: Forward and backward propagation in a simple neural network (1 hidden layer)

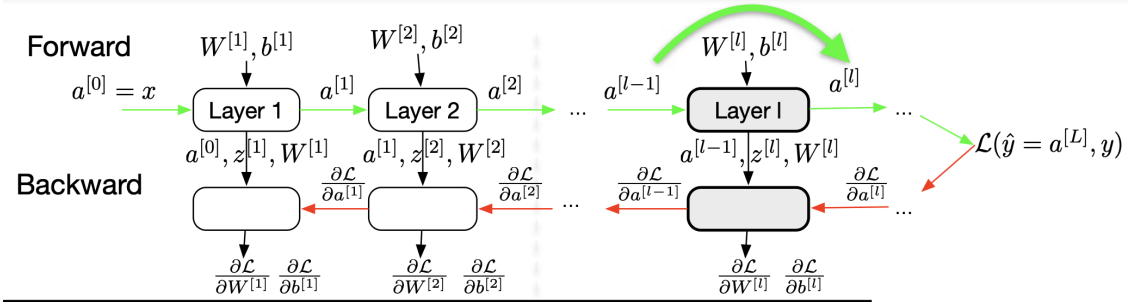


Figure 6: Forward and backward propagation schemes for l hidden layers

22.1.2 Loss Functions

The **loss function** measures the discrepancy between the prediction $\hat{y}$ and the target y. Training aims to minimize the empirical loss over the training set.

Mean Squared Error (MSE): MSE is commonly used for **regression** problems, typically with a single output neuron using a linear activation:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

For a single observation:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

The factor $\frac{1}{2}$ is often introduced because it simplifies the derivative:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y$$

Binary Cross-Entropy (BCE): BCE is used for **binary classification**. The output layer generally contains one neuron with a sigmoid activation:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The binary cross-entropy loss is:

$$\mathcal{L}_{\text{BCE}} = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

For N observations:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)\right]$$

Minimizing BCE is equivalent to maximizing the likelihood of the observed labels under the Bernoulli model.

Categorical Cross-Entropy: For **multi-class classification**, the output layer generally uses a softmax activation for K classes:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

For one-hot encoded targets:

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

For a dataset of N observations:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log(\hat{y}_{ik})$$

Summary:

Task	Output activation	Typical loss
Regression	Linear	MSE
Binary classification	Sigmoid	BCE
Multi-class classification	Softmax	Cross-Entropy

22.1.3 Activations Functions

Activation functions introduce **non-linearity** into neural networks. Without non-linear activation functions, a stack of linear layers would still be equivalent to a single linear transformation.

Sigmoid

The sigmoid function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

The maximum derivative is:

$$\max_x \sigma'(x) = \frac{1}{4} = 0.25$$

Therefore, when sigmoid activations are repeatedly used in deep networks, gradients can become very small during backpropagation. This is known as the **vanishing gradient problem**.

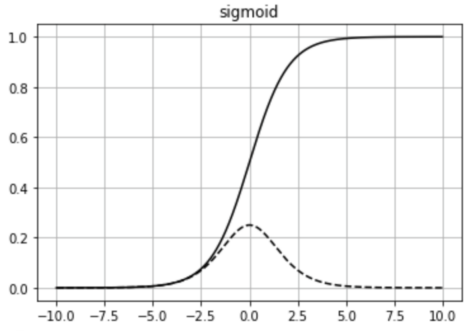


Figure 7: Sigmoid activation function and its derivative

Vanishing gradient problem

During backpropagation, gradients are obtained by multiplying many derivatives:

$$\frac{\partial \mathcal{L}}{\partial z^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[L]}} \prod_{k=l}^{L-1} \frac{\partial z^{[k+1]}}{\partial z^{[k]}}$$

If most of these derivatives have an absolute value smaller than 1, repeated multiplication can make the gradient exponentially small as it propagates through the network.

For sigmoid, the derivative is always bounded by 0.25. Consequently, a deep network containing many sigmoid layers can suffer from very small gradients in early layers.

This makes learning slow or even prevents early layers from receiving meaningful updates.

Tanh

The hyperbolic tangent is:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Its derivative is:

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$

Unlike sigmoid, tanh is centered around zero:

$$\tanh(x) \in [-1, 1]$$

This generally makes optimization easier than with sigmoid. However, tanh still saturates for large positive or negative values, causing its derivative to approach zero. Therefore, it can still suffer from the vanishing gradient problem.

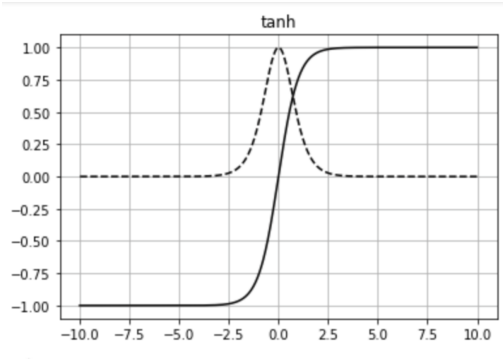


Figure 8: Hyperbolic tangent activation function

ReLU

The Rectified Linear Unit (ReLU) is defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Its derivative is:

$$\mathrm{ReLU}'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$

ReLU largely alleviates the vanishing-gradient problem on the positive side because its derivative is equal to 1 for positive inputs.

However, ReLU introduces the **dying ReLU problem**: if a neuron consistently receives negative inputs, its output becomes zero and its gradient is also zero. Such a neuron may stop learning.

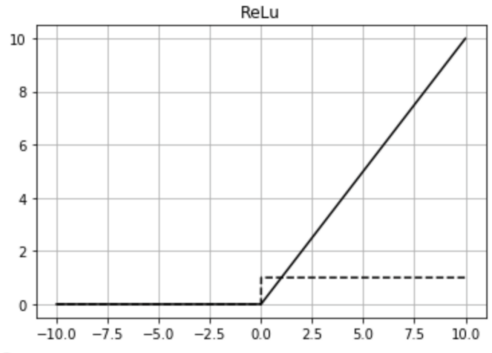


Figure 9: ReLU activation function.

ReLU variants

Several variants have been proposed to address limitations of ReLU:

- **Leaky ReLU**: keeps a small non-zero slope for negative inputs (fixed α):

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

- **Parametric ReLU (PReLU)**: similar to Leaky ReLU, but α is learned during training
- **ELU**: uses an exponential function for negative inputs and can produce smoother behavior around zero
- **SELU**: designed to encourage self-normalizing behavior under specific architectural and initialization conditions
- **GELU**: a smooth activation function widely used in modern neural-network architectures

Softmax
Softmax is generally used in the output layer for multi-class classification. It converts logits into a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

The outputs satisfy:

$$0 \leq \hat{y}_i \leq 1, \qquad \sum_{i=1}^{K} \hat{y}_i = 1$$

22.1.4 Training method

The objective of training is to find parameters θ that minimize the loss:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

Gradient-based methods update the parameters in the opposite direction of the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

where η is the learning rate.

Batch Gradient Descent
The gradient is computed using the entire training dataset before performing an update:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}_i(\theta_t)$$

- Advantages: deterministic gradient estimate and stable updates
- Disadvantages: expensive for large datasets, each parameter update requires processing the entire datase

Mini-Batch Gradient Descent
Instead of using the entire dataset, the gradient is estimated using a mini-batch of B samples:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta_t)$$

Mini-batch training is the standard approach for training neural networks because it provides a compromise between the efficiency of batch training and the frequent updates of stochastic training.

Stochastic Gradient Descent
In pure stochastic gradient descent (SGD), the gradient is estimated using a single randomly selected training example:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_i(\theta_t)$$

The stochastic gradient introduces noise into the optimization process. This can sometimes help the optimization process escape flat regions, saddle points, or poor local regions of the loss landscape.

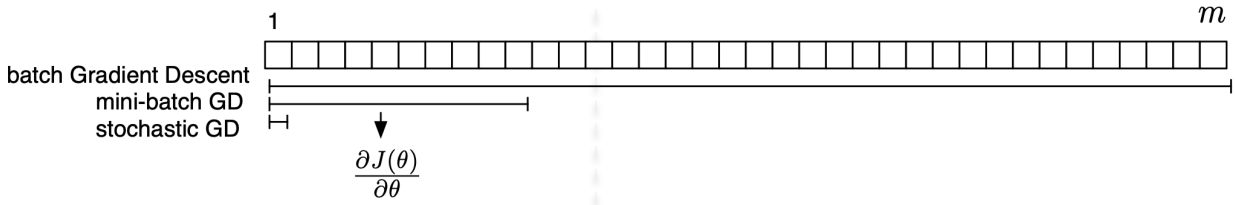


Figure 10: Comparison of batch, mini-batch, and stochastic gradient descent

Optimization difficulties
Training neural networks presents several optimization challenges:

- The loss function of a neural network is generally **highly non-convex**
- The loss landscape can contain many local minima, saddle points, and large flat regions
- Choosing an appropriate learning rate can be difficult
- A learning rate that is too small leads to slow convergence, while a learning rate that is too large can cause unstable training
- Different parameters may benefit from different effective learning rates, particularly when dealing with sparse features

In high-dimensional neural networks, saddle points and flat regions are often more important practical optimization issues than simply being trapped in poor local minima.

22.1.5 First-Order Optimization Methods

First-order optimization methods use the gradient of the loss function to determine the direction of parameter updates.

Momentum

Momentum introduces an exponentially decaying moving average of previous gradients:

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta v_t$$

The momentum term has two important effects:

- it reinforces updates when successive gradients point in similar directions
- it reduces oscillations when gradients repeatedly change direction

Momentum can therefore accelerate convergence, especially in directions where the optimization landscape is poorly conditioned.

Nesterov Accelerated Gradient (NAG)

Nesterov momentum evaluates the gradient at an approximate future position:

$$v_t = \beta v_{t-1} + \eta \nabla_\theta \mathcal{L}(\theta_t - \beta v_{t-1}), \qquad \theta_{t+1} = \theta_t - v_t$$

The main difference from classical momentum is that the gradient is evaluated after moving in the direction suggested by the previous momentum.

Adagrad

Adagrad adapts the learning rate independently for each parameter. It accumulates squared historical gradients:

$$G_t = G_{t-1} + \nabla_\theta \mathcal{L}(\theta_t) \odot \nabla_\theta \mathcal{L}(\theta_t)$$

where $\odot$ denotes element-wise multiplication. The update is:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla_\theta \mathcal{L}(\theta_t)$$

Parameters with frequently large gradients receive smaller updates, while parameters with small or infrequent gradients receive relatively larger updates. A major limitation is that the accumulated squared gradients continually increase, causing the effective learning rate to eventually become extremely small.

RMSprop

RMSprop addresses this issue by using an exponentially weighted moving average of squared gradients:

$$s_t = \beta s_{t-1} + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t) \odot \nabla_\theta \mathcal{L}(\theta_t)$$

The parameter update is:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

Adam

Adam combines ideas from momentum and adaptive learning-rate methods. It maintains estimates of the first and second moments of the gradients:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L}(\theta_t), \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\nabla_\theta \mathcal{L}(\theta_t) \odot \nabla_\theta \mathcal{L}(\theta_t)$$

Because these estimates are initialized at zero, bias-corrected estimates are used:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

The Adam update is:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

22.1.6 Normalization

During training, the distribution of the inputs received by a layer can change as the parameters of previous layers are updated. This phenomenon is often referred to as **internal covariate shift** in the original motivation for Batch Normalization.

For a layer l, its input is: $a^{[l-1]}$

Since $a^{[l-1]}$ depends on the parameters of the previous layer, its distribution can evolve during training.

Batch Normalization

Batch Normalization normalizes the pre-activation values within a mini-batch. For a mini-batch, compute:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu_B)^2$$

The normalized value is:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

The network then learns a scale and a shift:

$$y_i = \gamma \hat{z}_i + \beta$$

The parameters γ and β are learned during training. The normalization is therefore not intended to force every layer permanently to have mean 0 and variance 1. The learned parameters γ and β allow the network to recover an appropriate scale and offset when beneficial.

A common configuration is:

$$z^{[l]} \rightarrow \text{BatchNorm} \rightarrow g^{[l]}$$

Normalization can improve optimization stability and often allows the use of larger learning rates.

Other normalization methods

Several other normalization methods have been proposed to address specific limitations of Batch Normalization:

- **Layer Normalization**: normalizes across the features of a single sample, rather than across a mini-batch. This is particularly useful for recurrent neural networks
- **Instance Normalization**: normalizes each feature map of a single sample independently. It is often used in style transfer and image generation tasks
- **Group Normalization**: divides the channels into groups and normalizes within each group. It is effective for small batch sizes and is often used in computer vision tasks

22.1.7 Weight Initialization

The initial values of the weights have a significant impact on optimization. Initializing all weights to zero is not appropriate for a multi-neuron layer because all neurons would receive identical gradients and remain symetric. Random initialization breaks this symetry. However, the initial values should not be arbitrarily large or small:

- **weights that are too small can lead to vanishing activations and gradients**
- **weights that are too large can lead to exploding activations and gradients**
- **inappropriate initialization can slow down or destabilize training**

The initialization strategy should therefore take into account the activation function and the number of incoming and outgoing connections.

Xavier/Glorot initialization

Xavier initialization is particularly suitable for approximately symetric activation functions such as tanh.

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

It aims to maintain a reasonable variance of activations and gradients across layers.

He initialization

He initialization is designed primarily for ReLU-type activations:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

It compensates for the fact that ReLU sets negative activations to zero.

Summary

Activation	Typical initialization
Sigmoid / Tanh	Xavier / Glorot
ReLU / Leaky ReLU	He

22.1.8 Regularisation methods and Dropout

Regularization methods reduce the risk of **overfitting** by constraining the effective complexity of the model.

L1 Regularization

L1 regularization adds the absolute values of the weights to the loss:

$$\mathcal{L}_{L1} = \mathcal{L} + \lambda \sum_j |w_j|$$

The absolute-value penalty encourages many weights to become exactly or approximately zero. Therefore, L1 regularization tends to produce **sparse models**.

L2 Regularization

L2 regularization adds the squared weights:

$$\mathcal{L}_{L2} = \mathcal{L} + \frac{\lambda}{2} \sum_j w_j^2$$

L2 regularization discourages large weights and tends to distribute information across many parameters. In many optimization settings, L2 regularization is associated with **weight decay**.

Elastic Net
Elastic Net combines L1 and L2 regularization:

$$\mathcal{L}_{Elastic} = \mathcal{L} + \lambda_1 \sum_j |w_j| + \frac{\lambda_2}{2} \sum_j w_j^2$$

It combines the sparsity-inducing behavior of L1 with the stabilizing effect of L2.

Dropout
Dropout is a stochastic regularization technique that randomly deactivates a subset of neurons during training. If m_i is a Bernoulli random variable:

$$m_i \sim \text{Bernoulli}(p)$$

then the neuron activation can be written as (p is the probability of retaining a neuron):

$$\tilde{a}_i = \frac{m_i a_i}{p}$$

The scaling by $1/p$ corresponds to **inverted dropout** and ensures that the expected activation remains approximately unchanged. During inference, dropout is disabled.

The intuition is that the network cannot rely too strongly on a particular neuron or small group of neurons, which encourages more robust representations.

DropConnect
DropConnect applies a similar idea to **weights rather than activations**. Individual connections are randomly removed during training for a binary mask M: $\tilde{W} = M \odot W$

- Dropout randomly removes neuron activations
- DropConnect randomly removes connections/weights

22.1.9 Computation Graph

A **computation graph** represents a mathematical computation as a directed graph.

- **Nodes** represent inputs, parameters, intermediate values, and outputs
- **Edges** represent dependencies between quantities
- Each node can compute a local derivative with respect to its inputs

Example
For example, consider:

$$y = (a + b)c$$

A computation graph decomposes this expression into elementary operations:

$$d = a + b, \qquad y = dc$$

During the forward pass, intermediate quantities such as d are computed and stored.
During the backward pass, the chain rule is applied locally:

$$\frac{\partial y}{\partial a} = \frac{\partial y}{\partial d}\frac{\partial d}{\partial a} = c \cdot 1 = c$$

Similarly:

$$\frac{\partial y}{\partial b} = c, \qquad \frac{\partial y}{\partial c} = d$$

The computation graph therefore provides a systematic way of performing automatic differentiation. An important advantage is that intermediate quantities are **computed once and reused**. This avoids repeatedly recomputing the same expressions and makes backpropagation computationally efficient. In modern deep-learning frameworks, automatic differentiation systems construct or trace such computational graphs and automatically compute gradients required by optimization algorithms.

22.2 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are neural network architectures specifically designed to process data with a grid-like structure, such as images, video frames, and some forms of spatial or temporal signals.

The main idea is to exploit two important properties of images:

- **locality**: nearby pixels are more strongly related than distant pixels
- **translation structure**: the same visual pattern can appear at different spatial locations

Instead of connecting every neuron to every input value as in an MLP, CNNs use small learnable filters (kernels) that are applied locally across the input.

22.2.1 From MLPs to CNNs

In a fully connected MLP, each neuron is connected to every neuron in the previous layer. For an image, this results in a very large number of parameters.

Furthermore, each neuron would learn a pattern associated with a particular set of input positions. A pattern appearing elsewhere in the image would require other parameters to detect it.

CNNs address these limitations using:

- **Sparse connectivity**: each neuron is connected only to a local region of the previous layer
- **Weight sharing**: the same filter is applied at every spatial location

A convolutional neuron therefore detects a particular local pattern regardless of where that pattern occurs in the image.

The spatially limited set of connections associated with a neuron is related to its **receptive field**.

22.2.2 Convolution

A convolutional layer can be viewed as:

$$\boxed{\text{convolution} + \text{bias} + \text{activation}}$$

More explicitly, with u is the input, w is the learnable filter, b is the bias, and g is the activation function.

$$z = u * w + b \qquad y = g(z)$$

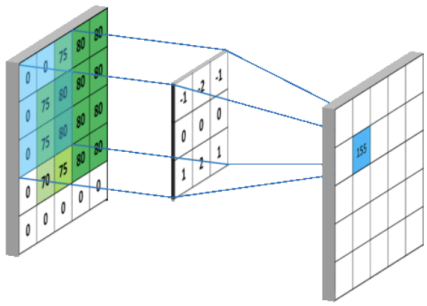


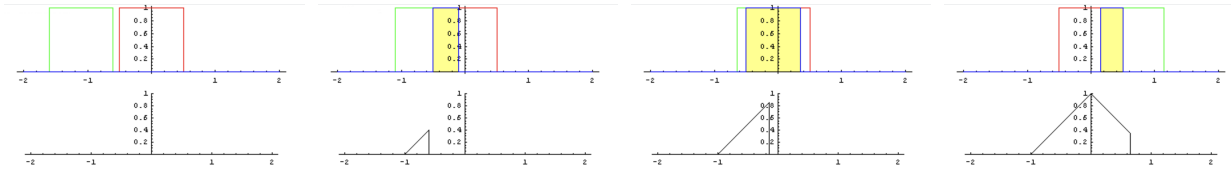
Figure 11: Convolution operation in a CNN layer: filter w is applied across the input u, producing the feature map z

Continuous convolution

For two continuous functions f and g, the convolution is defined as:

$$h(t) = (f * g)(t) = \int_{-\infty}^{+\infty} f(\tau) g(t - \tau)\, d\tau$$

The convolution combines two functions by sliding one function over the other and computing their overlap.



(a) Function f (b) Function g (c) Convolution $h = f * g$ (d) Convolution $h = f * g$

Figure 12: Illustration of continuous convolution: $h(t)$ is computed by sliding g over f and integrating their product

The classical properties of convolution include:

- **Commutativity**: $f * g = g * f$
- **Associativity**: $f * (g * h) = (f * g) * h$
- **Distributivity**: $f * (g + h) = f * g + f * h$
- **Identity**: $f * \delta = f$, where δ is the Dirac delta function
- **Differentiation**: $\frac{d}{dt}(f * g) = \frac{df}{dt} * g = f * \frac{dg}{dt}$
- **Convolution theorem**: $\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$, where $\mathcal{F}$ denotes the Fourier transform

Discrete convolution

For discrete signals:

$$h[n] = (f * g)[n] = \sum_{i=-\infty}^{+\infty} f[i] g[n - i]$$

In a CNN, the operation is usually implemented as a **cross-correlation** rather than a mathematically strict convolution, because the kernel is generally not flipped:

$$(u * w)[x] = \sum_i u[x+i]w[i]$$

22.2.3 2D Convolution for Images

An image is naturally represented as a two-dimensional signal. A 2D convolution can therefore be written as:

$$(f * g)(s,t) = \sum_i \sum_j f(i,j)g(s-i, t-j)$$

In CNNs, the kernel parameters are usually denoted by w:

$$z(y,x) = \sum_i \sum_j u(i,j)w(y-i, x-j)$$

The filter weights determine which type of local pattern the convolutional neuron responds to: horizontal or vertical edges, corners, textures, color transitions, simple shapes.

As the network becomes deeper, later filters can combine simpler features into increasingly complex structures.

22.2.4 Why CNNs Instead of MLPs?

CNNs exploit two fundamental properties.

Sparse connectivity

Suppose an image contains $m \times n$ pixels and a convolutional kernel has size $k \times k$, with $k \ll m, n$. A convolutional neuron only sees k^2 input values instead of all mn pixels. This dramatically reduces the number of connections.

For example, an MLP neuron connected to a 224×224 RGB image would require $224 \times 224 \times 3 = 150\,528$ weights for a single neuron. A 3×3 convolutional filter only requires $3 \times 3 \times 3 = 27$ weights per output channel, ignoring the bias.

Weight sharing

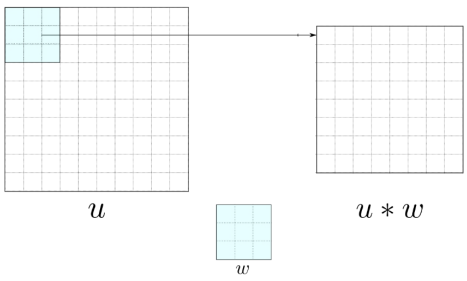
The same kernel is applied at every spatial location. Therefore, instead of learning a different set of parameters for every position, the network learns one set of parameters that detects the same type of pattern everywhere. This gives CNNs an important form of **translation equivariance**: translating the input generally translates the resulting feature map. This does not mean that the entire CNN is automatically translation invariant. Pooling, striding, padding, and later architectural components can modify this behavior.

22.2.5 Borders and Padding

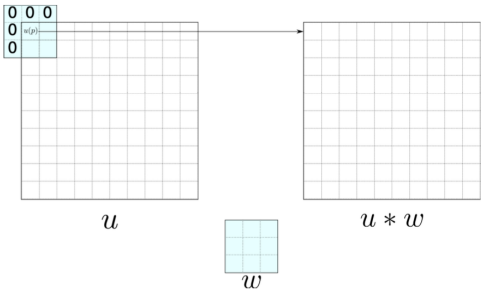
When a kernel is applied near the border of an image, part of the kernel may extend outside the input. Two common approaches are **valid** and **same** padding:

- **Valid convolution: no additional values are added around the image.** The kernel is only applied where it completely fits inside the input. For a one-dimensional input of size m and kernel size k, with stride 1: $m_{\text{out}} = m - k + 1$. More generally, for an input of size m and kernel size k, with stride s: $m_{\text{out}} = \lfloor \frac{m-k}{s} \rfloor + 1$.
- **Same convolution: values are added around the border, usually zeros.** For stride 1, the padding is chosen so that the output has approximately the same spatial dimensions as the input. For an odd-sized kernel and symetric padding: $p = \frac{k-1}{2}$, thus $m_{\text{out}} = m$.

A **stride** determines how far the kernel moves at each step. A stride of 1 means the kernel moves one pixel at a time, while a stride of 2 means it moves two pixels at a time, effectively downsampling the output.



(a) Valid convolution: reduces output size, no padding is added



(b) Same convolution: padding is added to preserve the spatial dimensions

22.2.6 Multiple Channels and Multiple Filters

Real images usually contain several channels. For example, an RGB image has $u \in \mathbb{R}^{H\times W\times 3}$. A convolutional layer generally uses several filters. Suppose a layer contains K filters: $\{w_1, w_2, \ldots, w_K\}$. Each filter produces one feature map. The feature maps are then stacked along the channel dimension: $u^{[l+1]} = [u * w_1,\, u * w_2,\, \ldots,\, u * w_K]$.

If the spatial dimensions are $m \times n$, the resulting activation has shape: $u^{[l+1]} \in \mathbb{R}^{m\times n\times K}$. If the input has K channels and the layer produces J output channels, each filter must span the entire input depth. If the spatial kernel has dimensions $s_H \times s_W$, the complete weight tensor has shape: $s_H \times s_W \times K \times J$. Therefore, the number of weights is $s_H s_W K J$.

For one output channel, the convolution is:

$$z(y, x, j) = \sum_i \sum_k \sum_c u(i, k, c) w_j(y - i, x - k, c) + b_j$$

The key point is that the filter mixes information from **all input channels** to generate one output feature map.

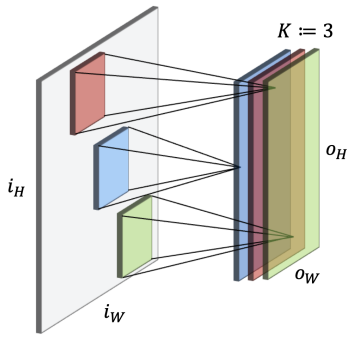


Figure 14: Multiple filters applied to a multi-channel input

22.2.7 Bias

A convolutional layer can also contain a bias. The important difference from an MLP is that there is typically **one bias per output feature map**, rather than one bias for every spatial neuron. If there are J output channels: $b \in \mathbb{R}^J$, the same bias b_j is therefore shared across all spatial positions of feature map j.

22.2.8 Backpropagation Through a Convolution

Backpropagation in a CNN follows the same chain-rule principle as in an MLP. The difference is that the same parameter is used at many spatial locations. Suppose a weight w_{ij} is shared across several positions. Its gradient is therefore the sum of its contributions at all positions where it is used:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \sum_{x,y} \frac{\partial \mathcal{L}}{\partial z(x, y)} \frac{\partial z(x, y)}{\partial w_{ij}}$$

This is an important consequence of **weight sharing**: a single parameter receives gradient contributions from all locations where the filter was applied. Similarly, each input pixel receives gradient contributions from all convolutional outputs whose receptive fields contain that pixel.

22.2.9 Downsampling and Shift Invariance

CNNs often reduce the spatial resolution of feature maps as the network becomes deeper. This can be achieved using: pooling, strided convolutions, or other downsampling operations.

Downsampling reduces the spatial resolution while increasing the effective receptive field of later neurons. A cascade of convolution, non-linearity, and downsampling can make the network less sensitive to small translations or local changes in the input. Strictly speaking, convolution provides **translation equivariance**, while pooling/downsampling can contribute to approximate **translation invariance** for classification tasks.

22.2.10 Subsampling

Subsampling reduces the number of spatial samples. For a one-dimensional signal, a simple subsampling operation with factor δ can be written as:

$$S_\delta(x)[t] = x[\delta t]$$

For example, with $\delta = 2$, every second sample is retained. Subsampling reduces computational cost and spatial resolution, while increasing the effective receptive field of subsequent layers.

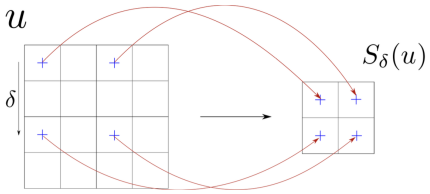


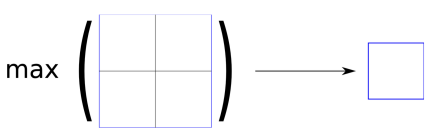
Figure 15: Subsampling operation

22.2.11 Max Pooling

Max pooling is a common downsampling operation. Given a pooling region, the output is the maximum value contained in that region:

$$S_{\max}(x) = \max_{i \in R} x_i$$

For a 2×2 pooling window with stride 2, each non-overlapping 2×2 region is reduced to a single value.



(a) Max pooling: a 2×2 region is reduced to its maximum value

(b) Max pooling with kernel size 2×2 and stride 2

During backpropagation, the gradient of max pooling is passed only through the input element that achieved the maximum value during the forward pass (determistic routing, no weights involved).

If $y = \max(x_1, x_2, x_3, x_4)$ and x_2 is the maximum, then $\frac{\partial y}{\partial x_2} = 1$ while the gradients with respect to the other inputs are zero. The position of the maximum therefore needs to be remembered during the forward pass.

22.2.12 Receptive Field

The **receptive field** of a neuron is the region of the original input that can influence its output. For a convolutional layer with a kernel of size $k \times k$, the receptive field of an output neuron corresponds to a local region of the previous layer. As layers are stacked, receptive fields grow. For example, two successive 3×3 convolutions with stride 1 have an effective receptive field of: 5×5 with respect to the original input.

Thus, deeper neurons can integrate information from increasingly large regions of the image. This allows the network to progress from:

$$\text{edges} \rightarrow \text{textures} \rightarrow \text{parts} \rightarrow \text{objects}$$

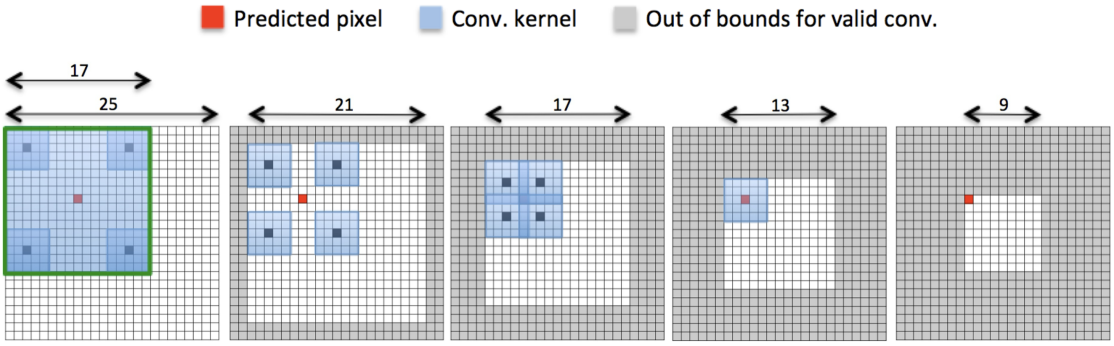


Figure 17: Growth of the receptive field through successive convolutional layers

An important distinction is:

- the **feature map size** depends on the input resolution and convolution parameters
- the **receptive field** describes how much of the original input can influence a particular neuron

A CNN can therefore produce predictions at many spatial locations, with each prediction initially depending only on its local receptive field rather than the entire input.

22.2.13 Output Size of a Convolution

For a convolution along dimension j, let: i_j: input size, k_j: kernel size, p_j: padding size, s_j: stride. The output size is given by:

$$\boxed{\sigma_j = \left\lfloor \frac{i_j - k_j + 2p_j}{s_j} \right\rfloor + 1}$$

For a 2D convolution, this formula is applied independently to the height and width dimensions. For example, for an input of 32×32, kernel 3×3, stride 1, and padding 1:

$$\sigma = \left\lfloor \frac{32 - 3 + 2}{1} \right\rfloor + 1 = 32$$

Therefore, the output remains 32×32.

22.2.14 Dilated Convolutions

A **dilated convolution**, also called an atrous convolution, introduces gaps between kernel elements. The dilation factor D controls the spacing between the kernel elements. A 2D dilated convolution can be written as:

$$z(y, x) = \sum_{i,j,c} u(y - Di, x - Dj, c)w(i, j, c)$$

When $D = 1$, the operation is a standard convolution. When $D > 1$, the kernel covers a larger spatial region without increasing the number of kernel parameters. For a $k \times k$ kernel, the effective kernel size is:

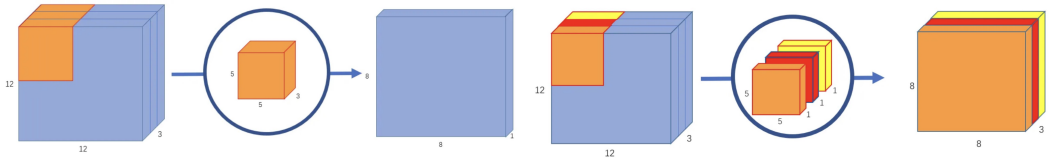
$$k_{\text{eff}} = k + (k - 1)(D - 1)$$

Thus, dilation increases the receptive field while keeping the number of learned parameters unchanged. Dilated convolutions are particularly useful in tasks requiring large spatial context, such as semantic segmentation.

22.2.15 Depthwise Convolutions

A standard convolution mixes information across both: spatial dimensions and input channels. In contrast, a **depthwise convolution** treats each input channel independently. If the input has K channels, a separate spatial filter is applied to each channel. For a kernel of size $k_H \times k_W$, there is $k_H k_W K$ parameters in total.

The number of output channels is typically equal to the number of input channels. In contrast, a standard convolution producing J output channels requires $k_H k_W K J$ parameters.



(a) Standard convolution: spatial and channel information are mixed simultaneously

(b) Depthwise convolution: each input channel is convolved indenpendently, preserving channel separation

22.2.16 Pointwise Convolutions

A **pointwise convolution** is a convolution with a 1×1 spatial kernel. It does not combine neighboring spatial positions. Instead, it performs a linear transformation of the channel vector at each pixel. For an input with K channels and J output channels: $W \in \mathbb{R}^{1\times1\times K\times J}$. At each spatial location (x, y): $z(x, y) = Wu(x, y) + b$.

Thus, a 1×1 convolution can:

- mix information between channels
- increase or decrease the number of channels
- introduce a learned linear transformation independently at every spatial position

22.2.17 Depthwise Separable Convolution

A **depthwise separable convolution** decomposes a standard convolution into two operations: depthwise convolution and pointwise 1×1 convolution. The depthwise convolution captures spatial information independently for each channel. The pointwise convolution then mixes the channels.

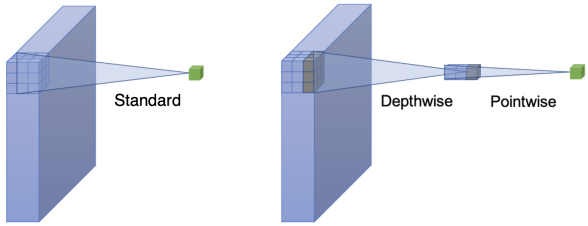


Figure 19: Depthwise separable convolution: depthwise convolution followed by a pointwise convolution

Consider: 64 input channels, 5×5 spatial kernels, 128 output channels

Ignoring biases, a standard convolution requires:

$$(64 \times 5 \times 5) \times 128 = 204\,800 \text{ parameters}$$

A depthwise separable convolution requires:

$$(5 \times 5) \times 64 + (64 \times 1 \times 1) \times 128 = 9\,792 \text{ parameters}$$

Therefore, the parameter reduction is substantial:

$$\frac{204\,800}{9\,792} \approx 20.9$$

The corresponding computational cost is also greatly reduced.

22.2.18 Important CNN Architectures

- **LeNet**: usefulness of convolutions for handwritten digit recognition
- **AlexNet**: first deep CNN to win the ImageNet competition, introduced ReLU activations, dropout, and data augmentation
- **VGG**: demonstrated that very deep networks with small 3×3 kernels can achieve excellent performance (even though computationally expensive due to the large number of parameters)
- **GoogLeNet (Inception)**: introduced the Inception module, which uses multiple filter sizes in parallel and concatenates their outputs, allowing the network to capture features at different scales
- **ResNet**: introduced residual connections to allow for training of very deep networks, addressing the vanishing gradient problem
- **U-Net**: image segmentation architecture with encoder-decoder structure and skip connections, enabling precise localization while capturing context:
  - **Encoder**: progressively reduce the spatial resolution while increasing the number of feature channels, capturing context and high-level features (downsampling path)
  - **Decoder**: progressively increase the spatial resolution to reconstruct a dense prediction map, to recover precise spatial localization (upsampling path)
  - **Skip connections**: connect feature maps directly to corresponding layers in the encoder and decoder, allowing the network to combine high-level features with low-level spatial information, improving segmentation accuracy (transfer of fine-grained details that may be lost during downsampling)

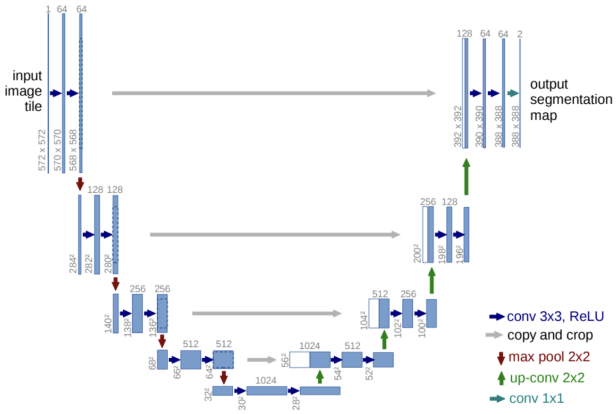


Figure 20: U-Net encoder-decoder architecture with skip connections

22.2.19 Residual Networks (ResNet)

As networks become deeper, optimization becomes increasingly difficult. ResNet introduced **residual connections** to facilitate the training of very deep networks. Suppose the desired underlying mapping is: $H(x)$.

- A standard network attempts to learn $H(x)$ directly
- A residual network instead learns a residual mapping: $F(x) = H(x) - x$

The original mapping can therefore be written as:

$$H(x) = F(x) + x$$

The block computes:

$$y = F(x) + x$$

The term x is passed through the **skip connection**. The identity path provides a direct route for information and gradients through the network, making optimization of very deep architectures easier.

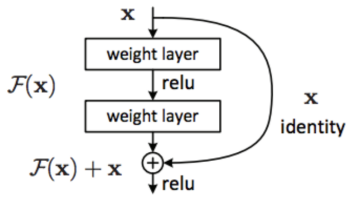


Figure 21: Residual connection in a ResNet block

22.2.20 CNNs for Motion Estimation

CNNs can also be used to estimate motion in video sequences. A common objective is to estimate the **optical flow**, which describes the apparent motion of pixels between two consecutive frames. Given two images:

$$I_t, \qquad I_{t+1}$$

A CNN can learn to predict a displacement fiel (u and v represent horizontal and vertical displacement):

$$F(x, y) = (u(x, y), v(x, y))$$

The network can be trained using pairs of frames for which ground-truth motion information is available. Learned motion representations can then be used in applications such as: action recognition, video segmentation, object tracking, video prediction.

FlowNet Frame 1 + Frame 2 → CNN feature extraction → feature combination → optical flow

22.2.21 Neural Style Transfer

CNN features can also be used to separate different types of visual information. In neural style transfer, an image is generated that combines the **content** of one image and the **style** of another image. A pre-trained CNN is used as a feature extractor. The content representation is obtained from intermediate CNN activations, while style can be represented using statistics of feature activations, commonly through Gram matrices. A typical objective combines:

$$\mathcal{L} = \lambda_C \mathcal{L}_{content} + \lambda_S \mathcal{L}_{style}$$

The generated image is then optimized so that its deep representation matches the desired content and style.

Figure 22: Neural style transfer using CNN representations

22.2.22 Understanding What CNNs Learn

One of the major challenges with deep CNNs is that their internal representations can be difficult to interpret. Although we know the mathematical operations performed by the network, it is not always obvious what semantic concepts are represented by individual neurons or feature maps. Several techniques can be used to investigate learned representations.

Deep Dream

Deep Dream modifies an input image in order to strongly activate selected features in a pre-trained CNN. The optimization can be described as:

$$x^* = \arg\max_x \mathcal{L}_{activation}(x)$$

where the objective encourages the network to increase the activation of selected neurons or feature maps. The resulting images often contain exaggerated or repeated patterns corresponding to features detected by the network.

Feature visualization

A more general approach is to find an input that maximizes the activation of a particular neuron or feature:

$$x^* = \arg\max_x \text{activation}_k(x)$$

In practice, regularization is often added to prevent the optimization from producing meaningless high-frequency patterns. Feature visualization can provide insight into what types of patterns individual layers or neurons respond to.

Adversarial examples
CNNs can be sensitive to carefully designed, very small perturbations of the input. Given an input x and perturbation δ, an adversarial example can be written as (δ is small according to a chosen norm):

$$x_{\text{adv}} = x + \delta$$

The objective can be formulated as finding a perturbation that changes the model's prediction while keeping the input visually similar:

$$\|\delta\|_p \leq \epsilon$$

For example, an attacker may solve an optimization problem of the form:

$$\max_{\delta} \mathcal{L}(x + \delta, y) \qquad \text{subject to} \qquad \|\delta\|_p \leq \epsilon$$

This demonstrates that high predictive accuracy on ordinary test images does not necessarily imply robustness to deliberately constructed perturbations. Adversarial phenomena can also be investigated in terms of feature representations: a network may produce strong activations for patterns that humans do not interpret as corresponding to the predicted object. Some perturbations can additionally be designed to transfer across multiple inputs or models, leading to the study of **universal adversarial perturbations**.

22.2.23 CNN Summary

Concept	Main idea
Convolution	Local feature extraction
Sparse connectivity	Each neuron sees only a local region
Weight sharing	Same filter is reused at every position
Feature map	Output produced by one filter
Padding	Controls spatial dimensions at the borders
Stride	Controls the spatial displacement of the kernel
Pooling	Downsamples feature maps
Max pooling	Keeps the maximum activation in each region
Receptive field	Input region influencing a neuron
Dilated convolution	Larger receptive field without more kernel parameters
Depthwise convolution	Spatial filtering independently per channel
Pointwise convolution	1×1 channel mixing
Depthwise separable convolution	Depthwise + pointwise convolution
Residual connection	Learns $F(x)$ and adds the input x
U-Net	Encoder-decoder architecture with skip connections
Optical flow	Dense representation of apparent image motion
Style transfer	Combines content and style representations
Feature visualization	Investigates learned representations
Adversarial examples	Small perturbations causing incorrect predictions

22.3 Sequential Data

Many data are naturally **ordered**: text, speech, audio, video, time series, DNA sequences, sensor logs, etc. In such data, the meaning of an element depends on the elements around it, and the length of the sequence may vary from one example to another. MLPs (fixed-size inputs) and CNNs (fixed grids) are not directly adapted to this setting, which motivates dedicated sequence models: RNNs, 1D convolutions, and Transformers.

22.3.1 Learning Paradigms

Training strategies distinguish the learning paradigm used to train a system (they are particularly relevant for sequence models, which are often trained on huge unlabeled corpora):

- **Semi-supervised learning**: a small amount of labeled data is combined with a larger amount of unlabeled data
- **Self-supervised learning**: when annotations are unavailable, a training objective is constructed directly from the data. Examples include next-token prediction, masked-token prediction, and contrastive methods such as SimCLR
- **Transfer learning**: a model is first pre-trained on a large dataset or task and then fine-tuned for a specific target task
- **Few-shot learning**: the model learns or adapts to a new task or class using only a small number of examples
- **Zero-shot learning**: the model performs a new task or recognizes new classes without having seen labeled examples of them during task-specific training, typically by exploiting previously learned representations or semantic information

- **Metric learning**: the model learns a representation space in which similar examples are close and dissimilar examples are far apart. It is often used for few-shot learning, retrieval, and recognition, using objectives such as contrastive loss or triplet loss

22.3.2 Representing Sequence Elements

As for MLPs and CNNs, the elements of a sequence can be of various types: real vectors (audio frames, sensor measurements), images (video frames), or **categorical values** (words, characters, tokens, notes).

Categorical values must be converted to vectors. For a vocabulary V of size $K = |V|$:

- **One-hot encoding**: the k-th word is the vector $e_k \in \{0,1\}^K$ with a single 1 at position k. It is very high-dimensional, sparse, and every pair of words is equally distant
- **Word embedding**: a learned matrix $E \in \mathbb{R}^{K \times d}$ maps each word to a dense vector $x = e_k^T E \in \mathbb{R}^d$ (row k of E), with $d \ll K$. Semantically similar words end up with close embeddings (see "Machine Learning Fundamentals & Applied Python / Transformers" in "Résumé Choses Faites" for more details on embeddings)

In practice, the input of a sequence model is therefore almost always a sequence of embedding vectors, while the output over the vocabulary is a softmax over K classes.

22.3.3 Notations

- **Inputs**: $x^{(t)}$ is the value of the input vector x at time t (of dimension n_x), and $\{x^{(1)}, x^{(2)}, \dots, x^{(T_x)}\}$ is a sequence of input vectors of length T_x
- **Outputs**: $y^{(t)}$ is the value of the output vector y at time t, and $\{y^{(1)}, y^{(2)}, \dots, y^{(T_y)}\}$ is a sequence of output vectors of length T_y
- **Training examples**: for the n-th example of the training set, we write $x^{\{n\}(t)}$ and $y^{\{n\}(t)}$, with lengths $T_x^{\{n\}}$ and $T_y^{\{n\}}$

The lengths of the input and output sequences, T_x and T_y, can be different, and can vary from one example to another.

22.3.4 Types of Sequence Problems

Depending on the task, a sequence model can take a single element or a sequence as input, and produce a single element or a sequence as output. When both the input and output are sequences, their lengths T_x and T_y may be equal or different.

Input x	**Output** y	**Type**	**Examples**
sequence $T_x > 1$	sequence $T_y = T_x$	Many-to-Many	Named entity recognition, POS tagging
sequence $T_x > 1$	single $T_y = 1$	Many-to-One	Sentiment analysis, video activity detection
single $T_x = 1$	sequence $T_y > 1$	One-to-Many	Text generation, music generation
sequence $T_x > 1$	sequence $T_y \neq T_x$	Many-to-Many	Speech recognition, machine translation

Many-to-Many ($T_x = T_y$)

An output must be produced for each element of the input sequence, so both sequences have the same length. In **Named Entity Recognition (NER)**, the network processes a sequence of words and predicts a label for each word (person, location, organization, none). Each input time step t is therefore associated with an output at the same time step. Part-of-Speech (POS) tagging works the same way (noun, verb, adjective, ...).

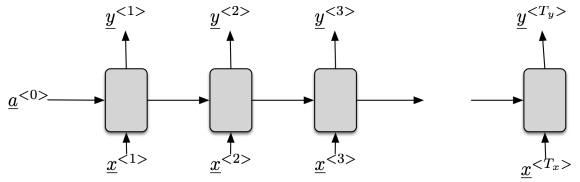


Figure 23: Many-to-Many with $T_x = T_y$: one output per input time step

Many-to-One ($T_x > 1$, $T_y = 1$)

The entire input sequence must be summarized into a single output. The network progressively processes the sequence, and its final representation is used to make a prediction about the sequence as a whole:

- in **sentiment analysis**, the input is a sequence of words and the output is a single label (positive or negative)
- in **action recognition**, a sequence of video frames is classified into a single activity

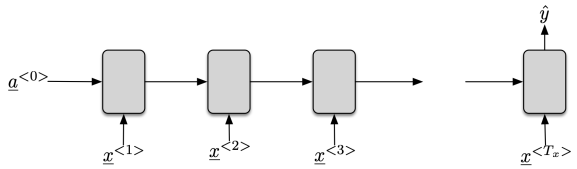


Figure 24: Many-to-One: the whole sequence is summarized into a single output

One-to-Many ($T_x = 1$, $T_y > 1$)
A single input is used to generate an entire sequence. The initial input conditions the generation process, while the network produces successive outputs over time. For example, a generative model can be conditioned on a single representation (a genre, an image, a start token) and then generate a sequence of words, code, or musical notes. Each generated output is usually fed back as the input of the next time step (**autoregressive** generation).

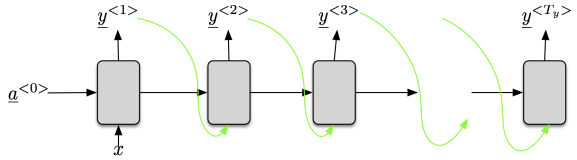


Figure 25: One-to-Many: a single input conditions the generation of a sequence

Many-to-Many ($T_x \neq T_y$)
Both the input and the output are sequences, but without the same length, so there is no direct one-to-one correspondence between input and output time steps:

- in **machine translation**, a sentence of T_x words may be translated into a sentence of T_y words
- in **automatic speech recognition**, a long sequence of acoustic frames is converted into a much shorter sequence of linguistic symbols

This configuration requires a mechanism that handles the absence of alignment between input and output time steps. It motivates the **encoder-decoder** architectures, and later **attention** mechanisms.

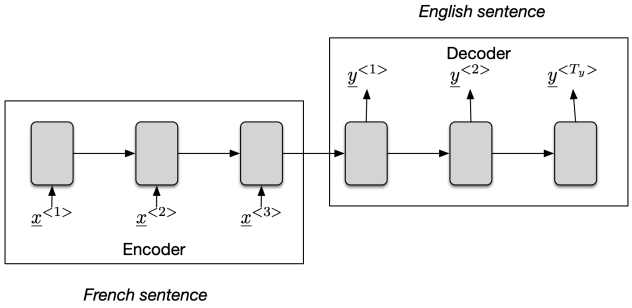


Figure 26: Many-to-Many with $T_x \neq T_y$: the input is first read entirely, then the output is generated

22.4 Recurrent Neural Networks (RNN)

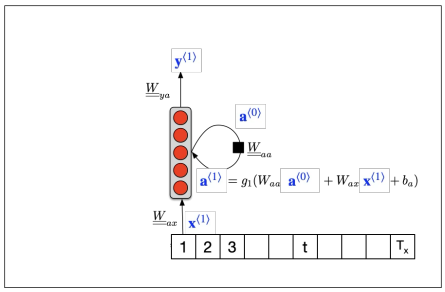
In an MLP, the hidden activation only depends on the current input. In an RNN, the hidden state $a^{(t)}$ depends not only on the input $x^{(t)}$ but also on the **hidden state at the previous time step** $a^{(t-1)}$. This dependency is made by adding a new learned projection, the **recurrence** term $W_{aa}a^{(t-1)}$. The hidden state therefore acts as a **memory** summarizing everything seen so far.

Nothing changes for the output $\hat{y}^{(t)}$: it is still only connected to the hidden state $a^{(t)}$, exactly as in an MLP:

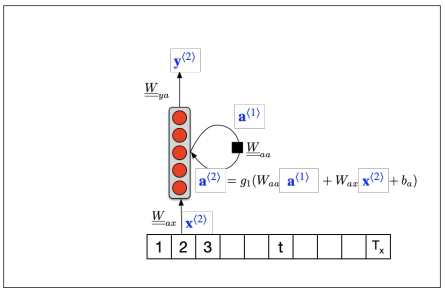
$$\boxed{a^{(t)} = g_a\left(W_{ax}x^{(t)} + W_{aa}a^{(t-1)} + b_a\right), \qquad \hat{y}^{(t)} = g_y\left(W_{ya}a^{(t)} + b_y\right)}$$

with $a^{(0)} = 0$ (or a learned initial state), g_a usually a tanh, and g_y chosen according to the task (softmax for classification, linear for regression). With n_a the hidden dimension, the shapes are $W_{ax} \in \mathbb{R}^{n_a \times n_x}$, $W_{aa} \in \mathbb{R}^{n_a \times n_a}$, $W_{ya} \in \mathbb{R}^{n_y \times n_a}$.

The matrices W_{ax}, W_{aa}, W_{ya} (and biases) are the parameters to be trained, like $W^{[1]}, W^{[2]}, \ldots$ in an MLP. The key difference is that they are **shared across all time steps**: the same cell is applied at every t, which allows the RNN to process sequences of any length with a fixed number of parameters (weight sharing in time, analogous to weight sharing in space for CNNs).

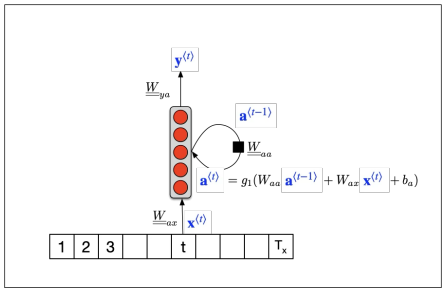


(a) $t = 1$

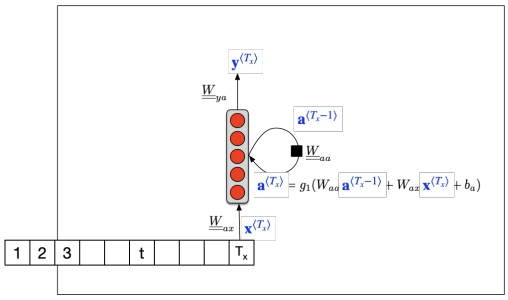


(b) $t = 2$

Figure 27: RNN unrolled through time: the same cell (same weights) is applied at every time step and passes its hidden state to the next one



(a) t



(b) $t = T_x$

22.4.1 Standard RNN (Vanilla RNN)

$a^{(t)}$ depends on the hidden states to its left: $a^{(t-1)}, a^{(t-2)}, \ldots, a^{(1)}$. At a given time t, the RNN only knows what comes from the left, i.e. the **past**. This is natural for online processing (prediction in real time), but limiting when the whole sequence is available: in "*Teddy* bears are on sale" vs. "*Teddy* Roosevelt was president", deciding whether *Teddy* is a person requires the words that follow.

22.4.2 Bidirectional RNN

A bidirectional RNN is made of two parallel and independent RNNs:

- a **left-to-right** RNN, with hidden states $\overrightarrow{a}^{(t)}$, which reads the sequence from $x^{(1)}$ to $x^{(T_x)}$
- a **right-to-left** RNN, with hidden states $\overleftarrow{a}^{(t)}$, which reads the sequence from $x^{(T_x)}$ to $x^{(1)}$

Each one has its own projection and recurrence matrices ($\overrightarrow{W}_{ax}, \overrightarrow{W}_{aa}$ and $\overleftarrow{W}_{ax}, \overleftarrow{W}_{aa}$):

$$\overrightarrow{a}^{(t)} = g_a\left(\overrightarrow{W}_{ax}x^{(t)} + \overrightarrow{W}_{aa}\overrightarrow{a}^{(t-1)} + \overrightarrow{b}_a\right), \qquad \overleftarrow{a}^{(t)} = g_a\left(\overleftarrow{W}_{ax}x^{(t)} + \overleftarrow{W}_{aa}\overleftarrow{a}^{(t+1)} + \overleftarrow{b}_a\right)$$

The prediction at time t uses the **concatenation** of both hidden states, so it depends on the past **and** the future of the sequence:

$$\hat{y}^{(t)} = g_y\left(W_{ya}\left[\overrightarrow{a}^{(t)}, \overleftarrow{a}^{(t)}\right] + b_y\right), \qquad W_{ya} \in \mathbb{R}^{n_y \times 2n_a}$$

The counterpart is that the complete sequence must be available before making any prediction: bidirectional RNNs are not suited to real-time processing.

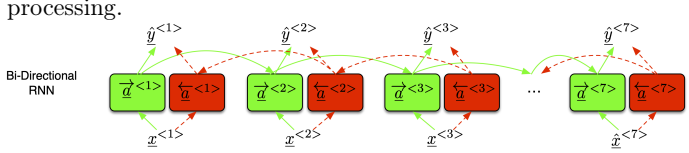


Figure 29: Bidirectional RNN: two independent RNNs read the sequence in opposite directions, their hidden states are concatenated

22.4.3 Deep RNN

Several recurrent layers can be stacked. The input of layer l at time t is made of:

- the value of the **previous layer at the same time** t: $a^{[l-1](t)}$ (with $a^{[0](t)} = x^{(t)}$)
- the value of the **same layer at the previous time** $t-1$: $a^{[l](t-1)}$

$$a^{[l](t)} = g\left(W_a^{[l]}\left[a^{[l](t-1)}, a^{[l-1](t)}\right] + b_a^{[l]}\right)$$

where $W_a^{[l]} = [W_{aa}^{[l]}, W_{ax}^{[l]}]$ simply stacks the recurrence and projection matrices of layer l. Each layer has its own parameters, shared across time. Unlike CNNs, deep RNNs usually remain shallow (2–3 recurrent layers), because each layer is already "deep" in time.

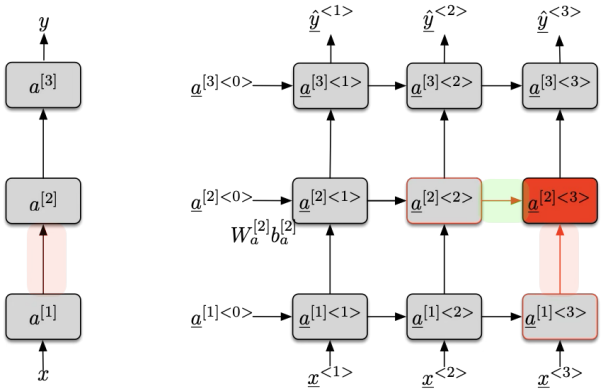


Figure 30: Deep RNN: each layer receives the previous layer at the same time and itself at the previous time

22.4.4 Training: Backpropagation Through Time (BPTT)

The **forward pass** goes from left to right: for $t = 1, \ldots, T$, the RNN computes $a^{(t)}$ and $\hat{y}^{(t)}$, providing an estimate at each time step. During training, the goal is to minimize the loss over the entire sequence:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}^{(t)}\left(\hat{y}^{(t)}, y^{(t)}\right)$$

The gradients are computed with **backpropagation through time (BPTT)**: the RNN is unrolled over time (it then becomes a very deep feed-forward network with shared weights), the chain rule is applied from right to left, and since the parameters are shared, the gradient contributions of all time steps are **accumulated**. The backward pass goes through the layers but also through time, so the vanishing gradient issue is even more severe than for an MLP.

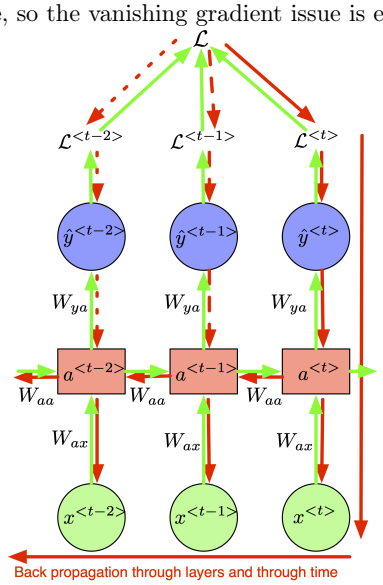


Figure 31: Backpropagation through time with forward and backward passes: the RNN is unrolled over time, and gradients are accumulated across time steps

Gradient with respect to W_{ya}

W_{ya} only acts on the output at time t, so each term is local:

$$\frac{\partial \mathcal{L}}{\partial W_{ya}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_{ya}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial W_{ya}}, \qquad \frac{\partial \hat{y}^{(t)}}{\partial W_{ya}} = g_y'\left(W_{ya} a^{(t)} + b_y\right) a^{(t)T}$$

Gradient with respect to W_{aa}

We still have $\frac{\partial \mathcal{L}}{\partial W_{aa}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_{aa}}$, but each term is no longer local: since $a^{(t)} = g_a(W_{ax}x^{(t)} + W_{aa}a^{(t-1)} + b_a)$, W_{aa} influences $a^{(t)}$ directly, and also indirectly through $a^{(t-1)}, a^{(t-2)}, \dots, a^{(1)}$, which all depend on W_{aa}. Applying the chain rule to the unrolled computation, and noting $\frac{\partial^+ a^{(k)}}{\partial W_{aa}}$ the **immediate derivative** (where $a^{(k-1)}$ is treated as a constant, and doesn't depend on W_{aa}), we get:

$$\frac{\partial \mathcal{L}^{(t)}}{\partial W_{aa}} = \frac{\partial \mathcal{L}^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial a^{(t)}} \left(\frac{\partial^+ a^{(t)}}{\partial W_{aa}} + \frac{\partial a^{(t)}}{\partial a^{(t-1)}} \frac{\partial^+ a^{(t-1)}}{\partial W_{aa}} + \dots + \frac{\partial a^{(t)}}{\partial a^{(t-1)}} \cdots \frac{\partial a^{(2)}}{\partial a^{(1)}} \frac{\partial^+ a^{(1)}}{\partial W_{aa}} \right)$$

$$\boxed{\frac{\partial \mathcal{L}^{(t)}}{\partial W_{aa}} = \frac{\partial \mathcal{L}^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial a^{(t)}} \sum_{k=1}^{t} \left(\prod_{i=k+1}^{t} \frac{\partial a^{(i)}}{\partial a^{(i-1)}} \right) \frac{\partial^+ a^{(k)}}{\partial W_{aa}}}$$

Gradient with respect to W_{ax}

Same issue: $\frac{\partial \mathcal{L}}{\partial W_{ax}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_{ax}}$, with exactly the same chain-rule structure (replace W_{aa} by W_{ax}).

Vanishing and exploding gradients

The source of the problem is the product of Jacobians. With $z^{(i)} = W_{ax}x^{(i)} + W_{aa}a^{(i-1)} + b_a$:

$$\frac{\partial a^{(i)}}{\partial a^{(i-1)}} = \operatorname{diag}\left(g_a'(z^{(i)})\right) W_{aa} \qquad \Rightarrow \qquad \left\| \prod_{i=k+1}^{t} \frac{\partial a^{(i)}}{\partial a^{(i-1)}} \right\| \leq (\gamma \|W_{aa}\|)^{t-k}, \quad \gamma = \max |g_a'|$$

As for MLPs, it requires the derivative of the activation function g_a (usually tanh, whose derivative is at most 1 and close to 0 when saturated), multiplied $t-k$ times:

- if $\gamma \|W_{aa}\| < 1$: the contribution of distant time steps decays **exponentially** (**vanishing gradient**). The network cannot learn **long-term dependencies**: a word at the start of a long sentence has almost no influence on the gradient at its end
- if the largest eigenvalue of W_{aa} is large: the gradient can grow exponentially (**exploding gradient**), causing unstable training (NaN values, huge jumps)

Practical solutions: **gradient clipping** against explosion (if $\|g\| > \tau$, then $g \leftarrow \tau\, g/\|g\|$), **truncated BPTT** (backpropagate over windows of limited length to reduce cost) and, against vanishing, **gated units** (LSTM, GRU).

22.4.5 Gated Units

The vanishing gradient problem in RNNs comes from repeatedly applying non-linearities and matrix multiplications across time. A solution is to create a **more direct path through time**, allowing information and gradients to propagate with fewer non-linearities and matrix multiplications. The amount of information that flows along this path is controlled by learned **gates**. This is the main idea behind gated RNNs, such as LSTMs and GRUs (and it is close in spirit to the skip connections of ResNet).

22.4.6 Long Short-Term Memory (LSTM)

The LSTM introduces two new ideas:

- a **memory cell** $c^{(t)}$ (same dimension n_a as the hidden state), which is transmitted across time steps
- **gates** Γ_*, which control the different processes depending on whether they are open or closed

As opposed to a standard RNN, which only transmits the hidden state $a^{(t)}$ over time, an LSTM transmits both the hidden state $a^{(t)}$ and the memory $c^{(t)}$, and the new hidden state $a^{(t)}$ is computed from the memory $c^{(t)}$, not directly from $a^{(t-1)}$.

Equations

- **Candidate memory** (it looks like the forward pass of a vanilla RNN):

$$\tilde{c}^{(t)} = \tanh\left(W_{ax}^{c} x^{(t)} + W_{aa}^{c} a^{(t-1)} + b^{c}\right)$$

- **Gates** $\Gamma_* \in [0,1]^{n_a}$: hard binary gates would not be differentiable (see the Perceptron), so they are implemented with **sigmoids**. They open or close depending on new projections of the input $x^{(t)}$ and of the previous hidden state $a^{(t-1)}$, which introduces new projection matrices to be trained. Each gate acts **individually on each dimension** of the memory:

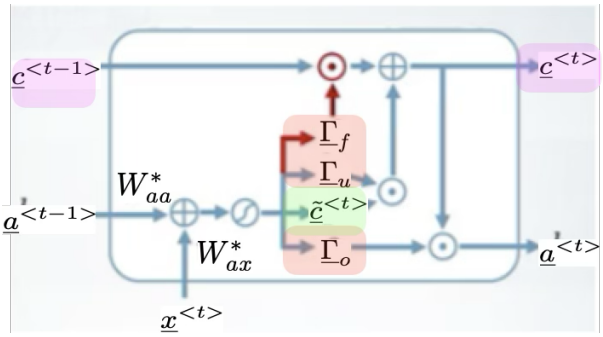
$$\Gamma_* = \sigma\left(W_{ax}^{*} x^{(t)} + W_{aa}^{*} a^{(t-1)} + b^{*}\right), \qquad * \in \{f, u, o\}$$

  - **forget gate** Γ_f: despite its name, it is actually a **keep-in-memory** gate (1 = keep, 0 = forget)
  - **update (input) gate** Γ_u: decides when to write the candidate into the memory
  - **output gate** Γ_o: decides when to output a value from the memory
- **Memory update**, from the candidate, the forget gate and the update gate:

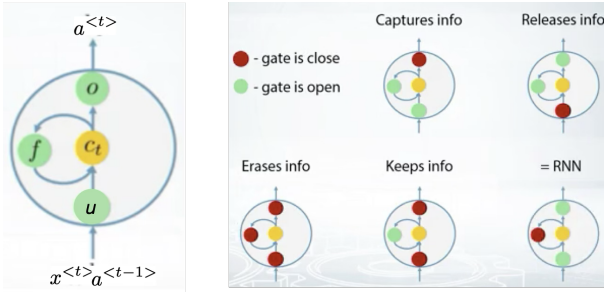
$$\boxed{c^{(t)} = \Gamma_f \odot c^{(t-1)} + \Gamma_u \odot \tilde{c}^{(t)}}$$

- **Hidden state**, from the memory and the output gate:

$$a^{(t)} = \Gamma_o \odot \tanh\left(c^{(t)}\right)$$



(a) LSTM cell



(b) LSTM regime: when gates are open and for what

Figure 32: LSTM: the memory cell $c^{(t)}$ runs along the top of the cell and is only modified by element-wise operations

Why it solves the vanishing gradient

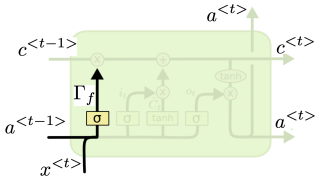
Along the memory path, $\frac{\partial c^{(t)}}{\partial c^{(t-1)}} = \text{diag}(\Gamma_f)$ (ignoring the dependence of the gates on $a^{(t-1)}$): there is no multiplication by W_{aa} and no squashing derivative. As long as the forget gate stays close to 1, the gradient flows almost unchanged across many time steps. The network itself **learns** when to keep and when to erase information.

Step-by-step process (with the intuition for language modelling, where the memory might encode the gender or number of the current subject)

1. What should be forgotten from the previous memory?

The forget gate answers for each component: 1 to keep it, 0 to forget it, a value in between to attenuate its influence.

Intuition: keep the gender of the current subject to predict the correct pronouns, or forget it when a new subject appears



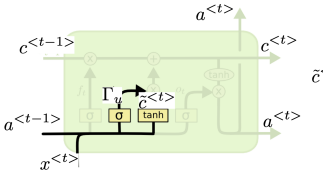
$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

Figure 33: Forget

2. What new information should be stored?

Compute the candidate $\tilde{c}^{(t)}$ and its contribution Γ_u.

Intuition: add the gender of the new subject, to replace the old one we are forgetting



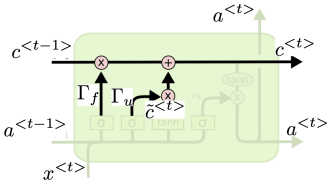
$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$
$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

Figure 34: Select new information

3. Write the new memory

Merge the old memory, filtered by the forget gate, with the new candidate, filtered by the update gate.

Intuition: actually drop the information about the old subject and refresh the memory with the new one



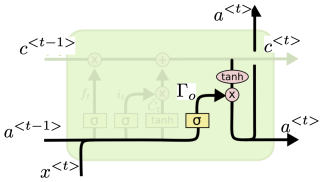
$$c^{<t>} = \Gamma_u \odot \tilde{c}^{<t>} + \Gamma_f \odot c^{<t-1>}$$

Figure 35: New memory

4. Write the new hidden state

Squash the memory with tanh and filter it by the output gate, to expose only the part that is useful now.

Intuition: having just seen a subject, output the information relevant for what comes next, e.g. whether it is singular or plural to conjugate the following verb



$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$
$$a^{<t>} = \Gamma_o \odot \tanh(c^{<t>})$$

Figure 36: New hidden state

22.4.7 Gated Recurrent Unit (GRU)

The GRU is a simplified version of the LSTM that follows the same main idea: using gates to control the propagation of information over time. The main differences are:

- the LSTM has two states, a hidden state $a^{(t)}$ and a separate memory $c^{(t)}$: the GRU has only **one state**, which plays the role of both ($a^{(t)} = c^{(t)}$)
- the LSTM uses three gates (forget, update, output): the GRU uses only **two gates** (update and relevance/reset)
- the forget and update decisions are **coupled**: what is written (Γ_u) is exactly what is forgotten ($1 - \Gamma_u$)
- the GRU therefore has fewer parameters and is computationally simpler

Both architectures aim at learning long-term dependencies by providing controlled paths for information and gradients through time. Neither is universally better: the LSTM is more expressive, the GRU faster and often as good on smaller datasets.

Equations (with $[\cdot,\cdot]$ the concatenation)

- **Relevance (reset) gate**: instead of using $c^{(t-1)}$ directly to compute the candidate, it is first filtered by Γ_r, which decides how much of the past is relevant for the candidate:

$$\Gamma_r = \sigma\left(W^r\left[c^{(t-1)}, x^{(t)}\right] + b^r\right)$$

- **Candidate memory**:

$$\tilde{c}^{(t)} = \tanh\left(W^c\left[\Gamma_r \odot c^{(t-1)}, x^{(t)}\right] + b^c\right)$$

- **Update gate**:

$$\Gamma_u = \sigma\left(W^u\left[c^{(t-1)}, x^{(t)}\right] + b^u\right)$$

- **Memory update and output**:

$$\boxed{c^{(t)} = (1 - \Gamma_u) \odot c^{(t-1)} + \Gamma_u \odot \tilde{c}^{(t)}, \qquad a^{(t)} = c^{(t)}}$$

When $\Gamma_u \approx 0$, the memory is copied unchanged ($c^{(t)} \approx c^{(t-1)}$), which creates the direct path through time.

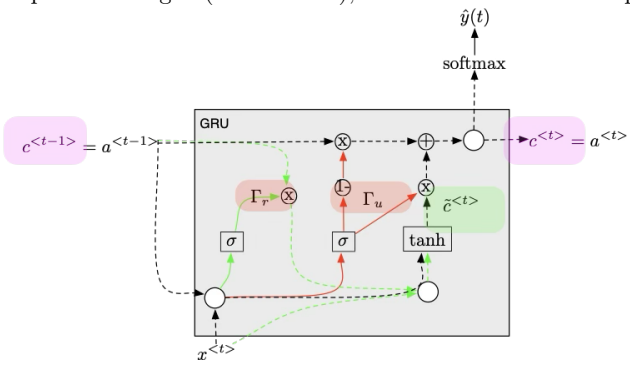


Figure 37: GRU cell: a single state, a relevance gate and an update gate

Cell	States transmitted	Gates	Parameters (per layer)
Vanilla RNN	$a^{(t)}$	none	$1\times$
LSTM	$a^{(t)}$, $c^{(t)}$	forget, update, output	$\approx 4\times$
GRU	$a^{(t)} = c^{(t)}$	relevance, update	$\approx 3\times$

22.4.8 One-to-Many: Autoregressive Language Models

A **language model** is a probabilistic model that assigns a (non-zero) probability to any sequence of words or tokens of a given vocabulary V. With the notation $W_1^L = w_1, w_2, \ldots, w_L$, the chain rule of probability gives the **autoregressive** factorization:

$$P(W_1^L) = P(w_1, w_2, \ldots, w_L) = \prod_{i=1}^{L} P\left(w_i \mid W_1^{i-1}\right), \qquad w_i \in V$$

$$P(W_1^L) = P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1, w_2) \cdots P(w_L \mid w_1 \ldots w_{L-1})$$

Historically, estimating $P(w_i \mid W_1^{i-1})$ directly from counts was impossible in practice, because the number of possible histories W_1^{i-1} grows exponentially: almost every long history is unseen in the training corpus (**data sparsity**). n-gram models truncated the history to the $n-1$ previous words. An RNN removes this limitation: its hidden state $a^{(t-1)}$ is a fixed-size summary of the **whole** history.

Training with teacher forcing

At each time step, the goal is to maximize the probability of observing the current word $y^{(t)} = w_t$, given:

- the previous word as input, $x^{(t)} = w_{t-1}$

- the hidden state $a^{(t-1)}$, which represents the context $w_1, \ldots, w_{t-2}$

The output $\hat{y}^{(t)}$ is a softmax over $K = |V|$ neurons giving the probability of each word, and $y^{(t)}$ is the one-hot encoding of the ground-truth word. In practice, we simply **delay the ground-truth sequence by one time step** to build the input sequence: $x^{(t)} = y^{(t-1)}$, starting from an empty context $a^{(0)} = 0$ and an empty (or start-of-sequence) input $x^{(1)} = 0$. For a training example, we minimize the categorical cross-entropy at each time step, summed over time:

$$\mathcal{L}_\theta\left(\hat{y}^{(t)}, y^{(t)}\right) = -\sum_{c=1}^{K} y_c^{(t)} \log \hat{y}_c^{(t)}, \qquad \mathcal{L}_\theta = \sum_t \mathcal{L}_\theta\left(\hat{y}^{(t)}, y^{(t)}\right) = -\sum_t \log P\left(w_t \mid W_1^{t-1}\right)$$

Minimizing this loss is equivalent to maximizing the likelihood of the training sentences. It is called **teacher forcing** because the network always receives the **true** previous word during training, never its own prediction. At inference time, the true words are not available: the network is fed its own sampled outputs, so errors can accumulate (**exposure bias**).

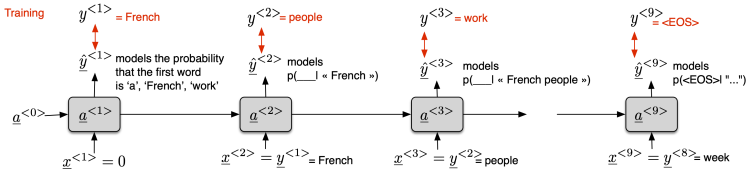


Figure 38: Teacher forcing: the input sequence is the ground-truth output sequence shifted by one time step

Generation and temperature

To generate text, we sample $w_t \sim P(w_t \mid W_1^{t-1})$, feed w_t as the next input, and repeat until an end-of-sequence token. A **temperature** T can be introduced by rescaling the output logits z before the softmax:

$$P(w_t = k) = \text{softmax}\left(\frac{z}{T}\right)_k = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}$$

- $T < 1$: sharper distribution, more deterministic and conservative ($T \to 0$: always the most probable word, i.e. greedy decoding)
- $T = 1$: original model distribution
- $T > 1$: flatter distribution, more diverse and random ($T \to \infty$: uniform)

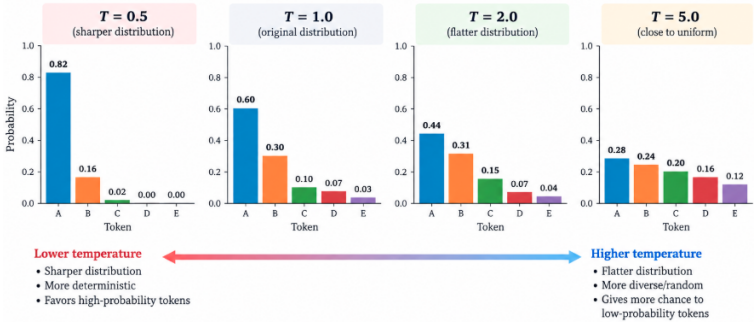


Figure 39: Effect of the temperature on the sampling distribution

22.4.9 Many-to-Many: Sequence-to-Sequence (Seq2Seq) Models

The goal is to transform an input sequence of length T_x in a domain X into an output sequence of length T_y in a domain Y, with in general $T_x \neq T_y$ (machine translation, speech recognition, text summarization). The model is made of two RNNs:

- an **encoder**, which reads the complete source sequence and produces a fixed-size representation of its content, the **context**
- a **decoder**, which generates the target sequence from this context

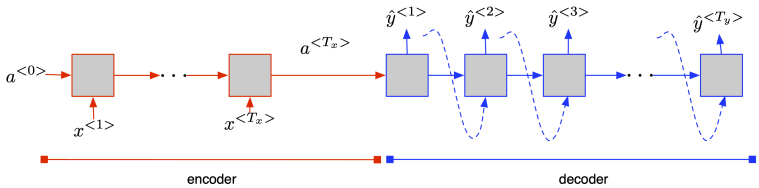


Figure 40: Seq2Seq encoder-decoder architecture

Example: machine translation

The training set contains pairs of sentences: a source sentence (English) $\{x^{(1)}, \dots, x^{(T_x)}\}$ and a target sentence (French) $\{y^{(1)}, \dots, y^{(T_y)}\}$. The encoder summarizes the source sentence into the context:

$$c = a^{(T_x)} = \text{Encoder}\left(x^{(1)}, \dots, x^{(T_x)}\right) \qquad \text{or} \qquad c = \left[\overrightarrow{a}^{(T_x)}, \overleftarrow{a}^{(1)}\right] \text{ for a bidirectional encoder}$$

The decoder behaves as a **conditional language model**: instead of modelling $p(y^{(1)}, \dots, y^{(T_y)})$, it models the probability of the target sentence **given** the source sentence. Using the chain rule:

$$p\left(y^{(1)}, \dots, y^{(T_y)} \mid x^{(1)}, \dots, x^{(T_x)}\right) = \prod_{t=1}^{T_y} p\left(y^{(t)} \mid y^{(1)}, \dots, y^{(t-1)}, x\right)$$

During training, we use teacher forcing: at each time step t, the decoder receives the true previous target token $y^{(t-1)}$ and predicts the next token $y^{(t)}$. The encoder and decoder are trained **jointly**, end-to-end, by minimizing the cross-entropy (equivalently, the negative log-likelihood):

$$\mathcal{L} = -\sum_{t=1}^{T_y} \log p\left(y^{(t)} \mid y^{(1)}, \dots, y^{(t-1)}, x\right)$$

The decoder can also be pre-trained independently as a language model $p(y^{(t)} \mid y^{(<t)})$ on a large corpus of target-language text, then connected to the encoder and fine-tuned so that it learns the conditional distribution $p(y^{(t)} \mid y^{(<t)}, x)$. At inference, the output is usually not sampled but searched to maximize the whole probability (greedy decoding or **beam search**, which keeps the B most probable partial sentences at each step).

Limitation and attention

The whole source sentence must be compressed into a single fixed-size vector c: this is a **bottleneck**, and translation quality drops for long sentences. **Attention** mechanisms remove it: instead of a single context, the decoder computes a **different context at each output step** τ, as a weighted average of **all** the encoder hidden states, where the weights depend on the current decoder state $s^{(\tau-1)}$:

$$e^{(\tau,t)} = \text{score}\left(s^{(\tau-1)}, a^{(t)}\right), \qquad \alpha^{(\tau,t)} = \frac{\exp e^{(\tau,t)}}{\sum_{t'=1}^{T_x} \exp e^{(\tau,t')}}, \qquad c^{(\tau)} = \sum_{t=1}^{T_x} \alpha^{(\tau,t)} a^{(t)}$$

The decoder can thus dynamically **focus** on the relevant source words for each generated word (e.g. on "cat" when generating "chat"). Taken to its limit, this idea leads to the Transformer, which keeps only attention and drops recurrence.

22.4.10 1D Convolution as an Alternative to RNNs

1D convolutions can also process sequential data. A 1D convolutional layer slides a set of filters along the **temporal dimension** of the input sequence (the channels being the dimensions of $x^{(t)}$):

$$z_j^{(t)} = \sum_{i=0}^{k-1} \sum_{c} w_j(i,c)\, x_c^{(t-i)} + b_j$$

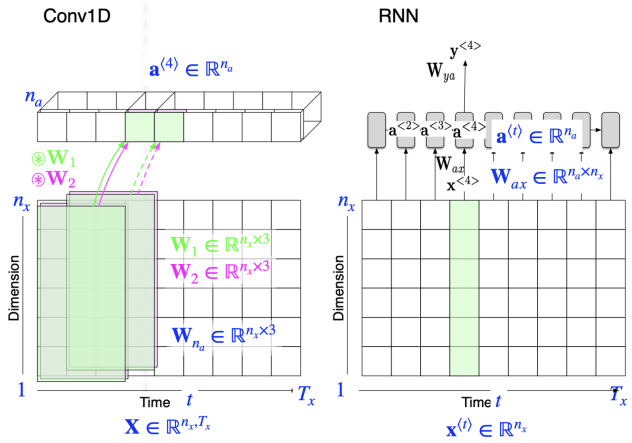


Figure 41: 1D convolution (parallel over time) vs. RNN (sequential over time)

Each filter captures local temporal patterns, and stacking layers lets the model learn dependencies over longer ranges, without any recurrent connection.

- **Parallelism**: all time steps are computed at once, whereas an RNN must compute $a^{(t-1)}$ before $a^{(t)}$ (sequential processing): training is much faster on GPUs
- **Stable gradients**: the gradient path length depends on the depth, not on the sequence length

- **Limited context**: the receptive field grows only linearly with depth ($L(k-1)+1$ for L layers): **dilated** convolutions make it grow exponentially (WaveNet, TCN)
- **Causal** convolutions (padding only on the left) ensure that $z^{(t)}$ only depends on the past, for generation and online prediction

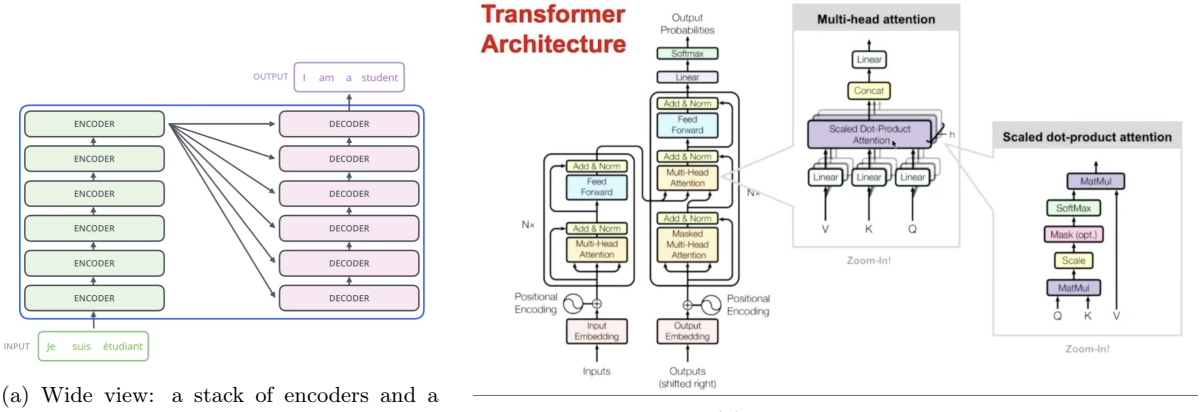
When to use which? 1D convolutions are well suited to local patterns and long signals (audio, sensor data, fast feature extraction), often as a first layer before an RNN. RNNs are natural for streaming data with unbounded context and constant memory per step. For long-range dependencies with parallel training, both have largely been replaced by Transformers.

22.5 Transformers

22.5.1 Full Architecture

Transformers are neural networks that processes sequences using **self-attention** rather than recurrence, models long-range dependencies while processing all sequence elements **in parallel**, which makes training much more efficient and forms the basis of most large language models (LLMs), and is also widely used for images, audio, speech, music, and multimodal models

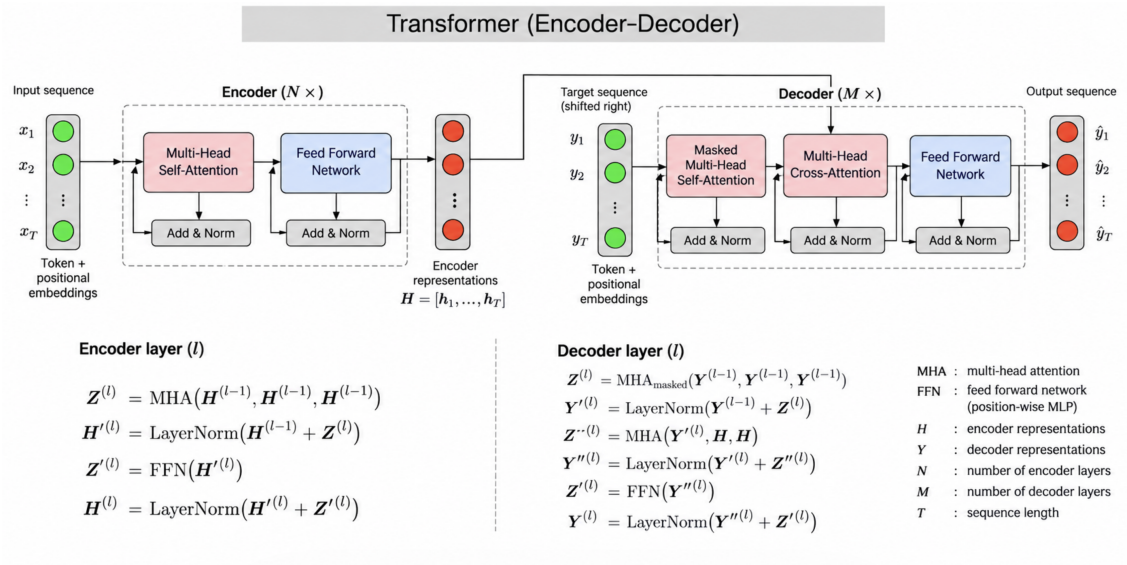
It remains an **encoder-decoder** architecture, built from four ingredients: attention mechanisms, in particular **self-attention**, **encoder blocks**, **decoder blocks**, **positional encoding**.



(a) Wide view: a stack of encoders and a stack of decoders

(b) Detailed architecture

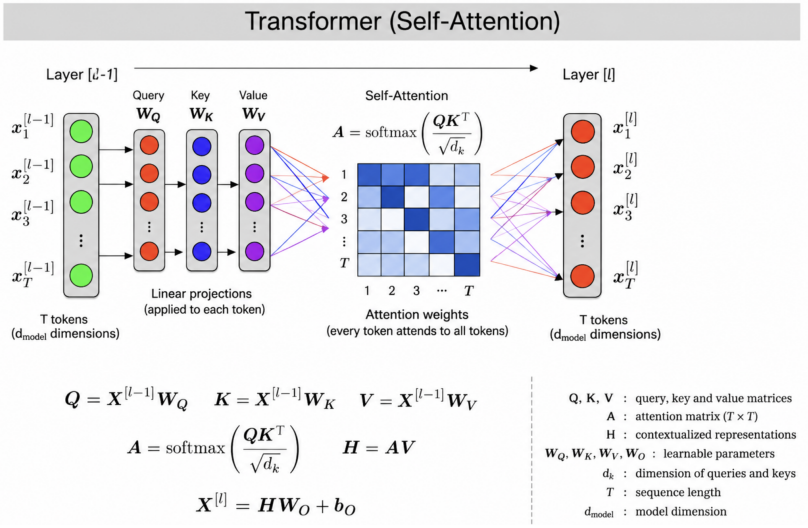
Figure 42: Transformer encoder-decoder architecture



22.5.2 Self-Attention Mechanism

Self-attention allows **each element of a sequence to attend to all the other elements** and to determine which ones are most relevant to build its own representation. For example, in "The animal didn't cross the street because *it*

was too tired", the representation of *it* should draw mostly on *animal*. It uses **Queries**, **Keys**, and **Values** to compute these interactions, and is the core mechanism of Transformers: it captures long-range dependencies in a single step and, unlike RNNs, processes all positions in parallel.



Vector formulation

For each input vector $x_i \in \mathbb{R}^{1\times d}$, three vectors are created by multiplication with three matrices $W^Q, W^K, W^V \in \mathbb{R}^{d\times d'}$, learned during training:

- the **query** $q_i = x_i W^Q$: what position i is looking for
- the **key** $k_j = x_j W^K$: what position j offers, compared with the queries
- the **value** $v_j = x_j W^V$: the information actually transmitted by position j

Usually $d' < d$ (in multi-head attention, $d' = d/h$, e.g. $512/8 = 64$). The attention weight α_{ij} answers: *while encoding position i (query), how much attention should be paid to position j (key)?*

- **Score**: dot product of the query q_i with the key k_j, divided by $\sqrt{d'}$:
$$e_{ij} = \frac{q_i k_j^T}{\sqrt{d'}}, \qquad (1, d')(d', 1)$$
- **Normalization** with a softmax over all keys:
$$\alpha_{ij} = \frac{\exp e_{ij}}{\sum_{l=1}^{T_k} \exp e_{il}}$$
- **Weighted sum of the values**: the values of the relevant positions are kept intact, the irrelevant ones are drowned out. The sum gives the output z_i at position i:
$$z_i = \sum_{j=1}^{T_k} \alpha_{ij} v_j$$

Why divide by $\sqrt{d'}$? If the components of q_i and k_j are independent with zero mean and unit variance, then $q_i k_j^T$ has variance d'. Large scores would saturate the softmax (almost one-hot weights), with tiny gradients. Dividing by $\sqrt{d'}$ brings the variance back to 1.

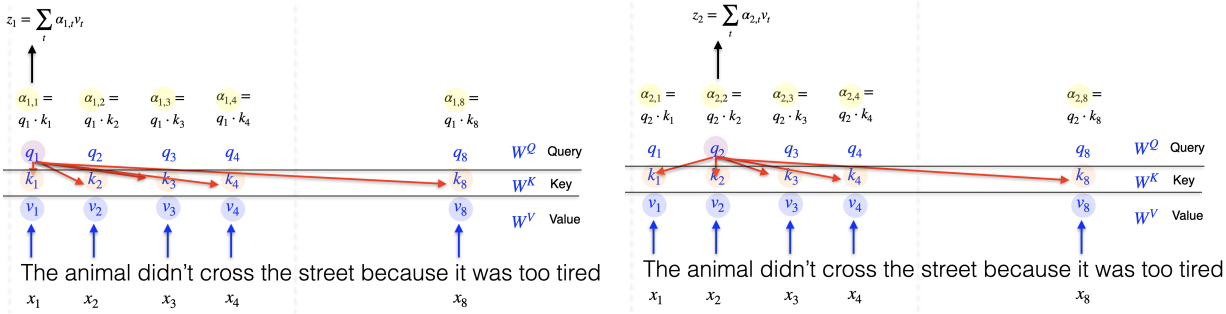


Figure 43: Step-by-step computation of self-attention for one position

Matrix formulation

All the computations can be expressed in matrix form, for all positions jointly. Stacking the inputs in $X \in \mathbb{R}^{T\times d}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \qquad (T, d') = (T, d)(d, d')$$

$$E = \frac{QK^T}{\sqrt{d'}} \quad (T_q, T_k) = (T_q, d')(d', T_k), \qquad A = \text{softmax}_{\text{rows}}(E), \qquad Z = AV \quad (T_q, d') = (T_q, T_k)(T_k, d')$$

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d'}}\right)V}$$

The whole computation is a few matrix products, which is why it is so efficient on GPUs. The price is the $T_q \times T_k$ attention matrix: the cost is **quadratic** in the sequence length.

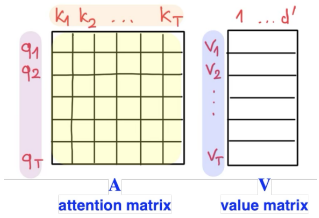


Figure 44: Matrix formulation of self-attention

Multi-head self-attention

A single attention head computes one kind of relation between positions. Several heads (often $h = 8$) are computed in parallel, each with its own matrices, so that each head can focus on a different type of relation (syntax, coreference, neighbouring words, ...):

$$Q_m = XW_m^Q, \quad K_m = XW_m^K, \quad V_m = XW_m^V, \qquad Z_m = \text{Attention}(Q_m, K_m, V_m) \in \mathbb{R}^{T_q \times d'}, \qquad m = 1, \ldots, h$$

The outputs of the heads are concatenated, then projected back to dimension d with a learned matrix W^O:

$$Z_+ = [Z_1, Z_2, \ldots, Z_h] \in \mathbb{R}^{T_q \times hd'}, \qquad Z = Z_+ W^O, \quad W^O \in \mathbb{R}^{hd' \times d}$$

With $d' = d/h$, multi-head attention costs about the same as a single head of full dimension.

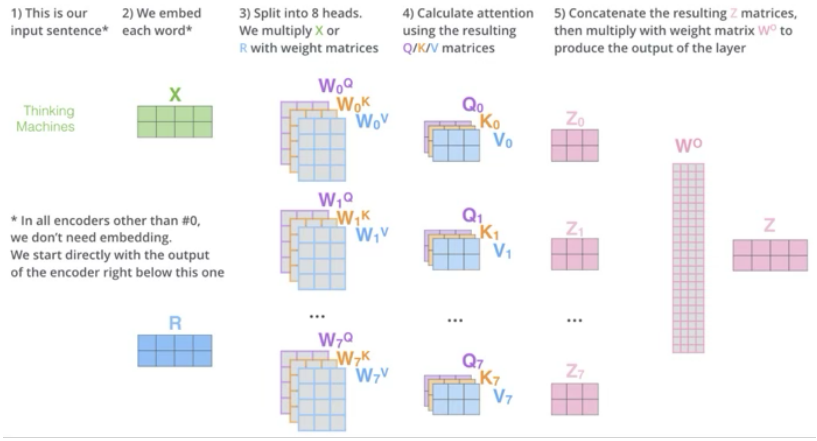


Figure 45: Multi-head self-attention: h heads computed in parallel, concatenated, and projected by W^O

22.5.3 Encoder Block

Input: $\{x_i\}_{i \in \{1,\ldots,T\}}$, the words turned into vectors by an embedding (plus positional encoding) for the bottom-most encoder, and the output of the previous encoder for the others. The output has the same shape (T, d), so blocks can be stacked. Each block has **two sub-layers**:

- **(1) Multi-head self-attention**: computes the dependencies between the inputs and outputs $\{z_i\}_{i \in \{1,\ldots,T\}}$. This is the only place where positions interact
- **(2) Position-wise feed-forward network** (hidden dimension $d_{ff} = 2048$ in the original model):

  $$\text{FFN}(z_i) = \max(0, z_i W_1 + b_1)W_2 + b_2$$

  It processes each z_i **independently**: the exact same network is applied to every position, so all positions can be processed in parallel

Each sub-layer is wrapped by a **residual connection** and a **layer normalization** ("Add & Norm"):

$$\text{output} = \text{LayerNorm}\,(x + \text{SubLayer}(x))$$

Layer normalization normalizes each vector across its **feature** dimension (instead of the batch dimension as in BatchNorm), which is independent of batch size and of sequence length. The residual connections, as in ResNet, allow very deep stacks to be trained. Many recent models place the normalization before the sub-layer (*pre-norm*: $x + \text{SubLayer}(\text{LayerNorm}(x))$), which is more stable for deep models.

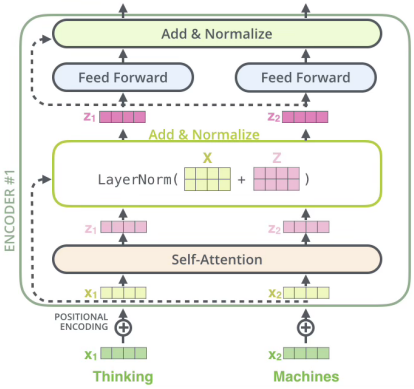


Figure 46: Encoder block: self-attention and feed-forward sub-layers, each with a residual connection and layer normalization

22.5.4 Decoder Block

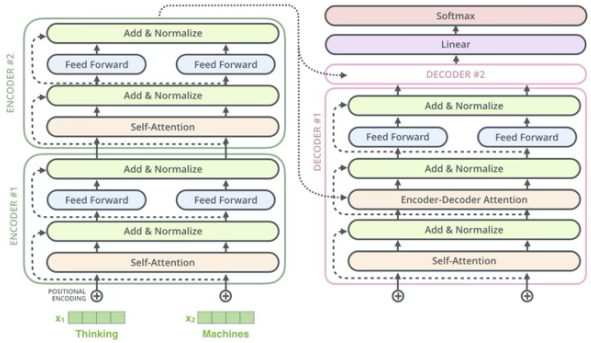
Each decoder block has **three sub-layers**, each with a residual connection and a layer normalization:

- **(1) Masked self-attention**, as in the encoder, except that each position is only allowed to attend to **earlier positions** of the output sequence (see next section)
- **(2) Encoder-decoder attention (cross-attention)**: helps the decoder focus on the relevant parts of the input sentence, like attention in Seq2Seq models. The queries come from the decoder, the keys and values from the output of the last encoder:
  - queries Q_{decoder}: *what information do I need to generate the current target token?*
  - keys K_{encoder}: *what information is available at each position of the source sentence?*
  - values V_{encoder}: *what information should I retrieve from each source position?*

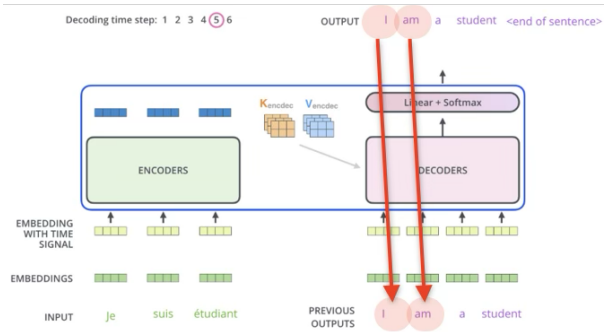
  The attention matrix has shape (T_y, T_x): one row per target position, one column per source position
- **(3) Feed-forward network**, as in the encoder

The output of the last decoder block goes through a **linear layer** (projection to the vocabulary size K) and a **softmax**, giving the probability of the next token.

Training vs. inference: during training, the whole target sequence is processed **in parallel** thanks to masking (teacher forcing) and during inference, the decoder must work **autoregressively** (slower): the token generated at each step is appended to the decoder input for the next step, until an end-of-sequence token is produced. The encoder, in contrast, runs only once on the source sentence.



(a) Link between the encoder output and the decoder blocks



(b) Autoregressive decoding during inference

Figure 47: Transformer decoder

22.5.5 Masked Self-Attention (Decoder)

The decoder is an autoregressive model: each output must be generated using only information from previous time steps. With teacher forcing, all time steps can still be trained **in parallel**, by feeding the decoder the shifted target sequence $\{\langle \text{BOS} \rangle, y_1, \ldots, y_{T-1}\}$ and using $\{y_1, y_2, \ldots, y_T\}$ as ground truth (exactly as for the RNN language model).

However, through self-attention, position i could directly look at $y_i, y_{i+1}, \ldots$ and simply copy the answer. We must ensure that no time step can access information from future tokens. This is done with **causal (masked) self-attention**: the scores of future positions are set to $-\infty$ **before** the softmax:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d'}} + M\right), \qquad M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Since $\exp(-\infty) = 0$, the attention weights of future positions are exactly zero, and the remaining weights still sum to 1. The mask must be applied in **every** decoder self-attention layer: otherwise, later layers could indirectly access future tokens through earlier ones. Note that only the decoder **self-attention** is masked: the cross-attention to the encoder is not, since the complete source sequence is available.

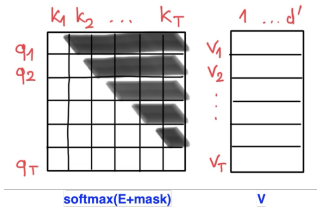


Figure 48: Causal mask: the black area (upper triangle) corresponds to masked scores, set to $-\infty$ before the softmax

22.5.6 Tokenization, Embedding and Positional Encoding

(see "Machine Learning Fundamentals & Applied Python / Transformers" in "Résumé Choses Faites" for more details)

- **Tokenization**: the text is split into tokens, usually **subwords** (e.g. BPE, WordPiece), which keeps the vocabulary of manageable size while handling rare and unseen words
- **Embedding**: each token index is mapped to a learned vector of dimension d (row of an embedding matrix $E \in \mathbb{R}^{|V| \times d}$)
- **Positional encoding**: self-attention is **permutation equivariant**, it does not encode the order of the elements by itself (permuting the input rows simply permutes the output rows: "dog bites man" and "man bites dog" would get the same representations). Position information is therefore added to the embeddings: $X \leftarrow X + P$, with $P \in \mathbb{R}^{T \times d}$ the positional encoding matrix

The original Transformer uses fixed **sinusoidal** encodings, with frequencies decreasing along the dimensions:

$$P_{\text{pos},2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \qquad P_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Each position gets a unique code, and the encoding of $\text{pos} + k$ is a linear function of that of pos, which makes relative positions easy to exploit. Alternatively, the positional encodings can simply be **learned** like the embeddings (BERT, ViT), or relative/rotary encodings can be used (RoPE, in most recent LLMs).

22.5.7 Transformer Families

- **Encoder-only** (BERT): bidirectional self-attention, trained with masked-token prediction, used for understanding tasks (classification, NER, retrieval)
- **Decoder-only** (GPT, most LLMs): masked self-attention, trained with next-token prediction, used for generation
- **Encoder-decoder** (original Transformer, T5): sequence-to-sequence tasks such as translation or summarization

All of them are pre-trained in a **self-supervised** way on huge corpora, then adapted by fine-tuning (transfer learning), or used directly in few-shot / zero-shot mode.

22.5.8 Comparison with RNNs and CNNs

For a sequence of length T, representation dimension d, and kernel size k:

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-attention	$O(T^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(Td^2)$	$O(T)$	$O(T)$
Convolutional	$O(kTd^2)$	$O(1)$	$O(\log_k T)$ (dilated)

The **maximum path length** is the number of steps a signal must travel between two arbitrary positions: the shorter it is, the easier long-range dependencies are to learn. Self-attention connects every pair of positions directly and is fully parallel, but its quadratic cost in T limits the context length (motivating efficient attention variants).

22.5.9 Application to Images: Vision Transformer (ViT)

The Transformer encoder can be applied to images by treating an image as a **sequence of patches**:

- **Patch decomposition**: an image $H \times W \times C$ is split into $N = \frac{HW}{P^2}$ non-overlapping patches of size $P \times P$ (e.g. 224×224 with $P = 16$ gives $N = 196$ patches)
- **Linear projection**: each flattened patch (P^2C values) is projected to dimension d, the equivalent of a word embedding
- **Class token**: a learnable [CLS] vector is prepended to the sequence, its final representation summarizes the image
- **Positional encoding**: learned position embeddings are added, otherwise the patch order (the image layout) would be lost
- **Transformer encoder**: a standard stack of encoder blocks
- **Classification head**: an MLP on the output of the [CLS] token

Comparison with CNNs: in a CNN, the receptive field grows progressively with depth, in a ViT, every patch can attend to every other patch from the first layer, so the receptive field is the **whole image** in every layer. On the other hand, a ViT has much weaker **inductive biases**: locality and translation equivariance are not built in, and must be learned from data. ViTs therefore need large datasets (or strong pre-training and data augmentation) to outperform CNNs, but scale better when data is abundant.

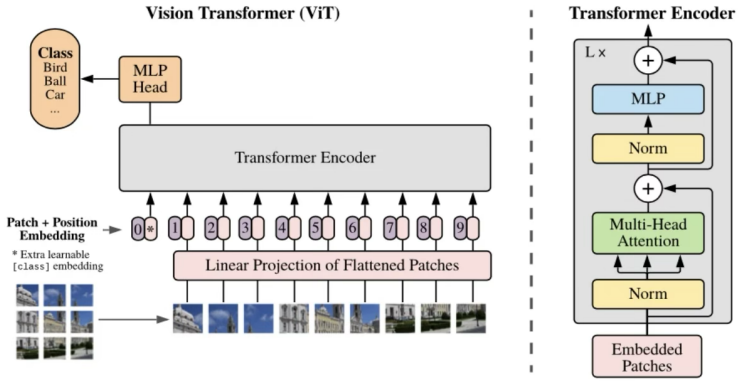


Figure 49: Vision Transformer: the image is split into patches, linearly embedded, combined with position embeddings, and processed by a Transformer encoder

22.5.10 Sequence Models Summary

Concept	Main idea
RNN	Hidden state carries the past: same weights at every time step
Bidirectional RNN	Two RNNs in opposite directions: uses past and future
Deep RNN	Stacked recurrent layers
BPTT	Backpropagation on the network unrolled through time
Vanishing / exploding gradient	Product of Jacobians over time: gradient clipping, gated units
LSTM	Memory cell + forget, update, output gates
GRU	Single state + update and relevance gates: lighter than LSTM
Language model	Autoregressive factorization $\prod_i P(w_i \mid W_1^{i-1})$
Teacher forcing	Ground-truth previous token as input during training
Temperature	Rescaled logits: controls diversity of sampling
Seq2Seq	Encoder summarizes the source, decoder generates the target
Attention	Context recomputed at each step as a weighted sum of all states
1D convolution	Parallel local temporal filtering: dilation for long context
Self-attention	$\text{softmax}(QK^T/\sqrt{d'})V$: every position attends to every other
Multi-head attention	Several heads in parallel, concatenated and projected
Encoder / decoder blocks	Attention + FFN, each with residual connection and LayerNorm
Masked self-attention	Causal mask: parallel training of an autoregressive decoder
Cross-attention	Queries from the decoder, keys and values from the encoder
Positional encoding	Injects order information into a permutation-equivariant model
ViT	Image as a sequence of patches processed by a Transformer encoder